IN3050/IN4050 Mandatory Assignment 2, 2025: Supervised Learning

Name: Justas Simutis

Username: justassi

Rules

Before you begin the exercise, review the rules at this website: https://www.uio.no/english/studies/examinations/compulsory-activities/mn-ifi-mandatory.html , in particular the paragraph on cooperation. This is an individual assignment. You are not allowed to deliver together or copy/share source-code/answers with others. Read also the "Routines for handling suspicion of cheating and attempted cheating at the University of Oslo": https://www.uio.no/english/studies/examinations/cheating/index.html We do not entirely prohibit the use of generative language models ("smart assistants" like ChatGPT, Llama, Claude or Copilot), but you must clearly acknowledge this at all times, following the UiO guidelines: https://www.uio.no/english/studies/resources/ai_student.html Note also that you must fully understand *all* the parts of you submissions, even if you got some help from a generative model. This will be tested during your peer review sessions (https://www.uio.no/studier/emner/matnat/ifi/IN3050/v25/Peer%20review/). By submitting this assignment, you confirm that you are familiar with the rules and the consequences of breaking them.

Delivery

Deadline: Friday, March 28, 2025, 23:59

Your submission should be delivered in Devilry. You may redeliver in Devilry before the deadline, but include all files in the last delivery, as only the last delivery will be read. You are recommended to upload preliminary versions hours (or days) before the final deadline.

What to deliver?

You are recommended to solve the exercise in a Jupyter notebook, but you might solve it in a regular Python script if you prefer.

Alternative 1

If you prefer not to use notebooks, you should deliver the code, your run results, and a PDF report where you answer all the questions and explain your work.

Alternative 2

If you choose Jupyter, you should deliver the notebook. You should answer all questions and explain what you are doing in Markdown. Still, the code should be properly commented. The notebook should contain results of your runs. In addition, you should make a PDF of your

solution which shows the results of the runs. (If you can't export: notebook -> latex -> pdf on your own machine, you may do this on the IFI linux machines.)

Here is a list of *absolutely necessary* (but not sufficient) conditions to get the assignment marked as passed:

- You must deliver your code (Python script or Jupyter notebook) you used to solve the assignment.
- The code used for making the output and plots must be included in the assignment.
- You must include example runs that clearly shows how to run all implemented functions and methods.
- All the code (in notebook cells or python main-blocks) must be runnable. If you have unfinished code that crashes, please comment it out and document what you think causes it to crash.
- You must also deliver a PDF of the code, outputs, comments and plots as explained above.

Your report/notebook should contain your name and username.

Deliver one single compressed folder (.zip, .tgz or .tar.gz) which contains your complete solution.

Important: if you weren't able to finish the assignment, use the PDF report/Markdown to elaborate on what you've tried and what problems you encountered. Students who have made an effort and attempted all parts of the assignment will get a second chance even if they fail initially. This exercise will be graded PASS/FAIL.

Goals of the assignment

The goal of this assignment is to get a better understanding of supervised learning with gradient descent. It will, in particular, consider the similarities and differences between linear classifiers and multi-layer feed forward neural networks (multi-layer perceptrons, MLP) and the differences and similarities between binary and multi-class classification.

Tools

The aim of the exercises is to give you a look inside the learning algorithms. You may freely use code from the weekly exercises and the published solutions. You should not use machine learning libraries like Scikit-Learn or PyTorch, because the point of this assignment is for you to implement things from scratch. You, however, are encouraged to use tools like NumPy, Pandas and MatPlotLib, which are not ML-specific.

The given precode uses NumPy. You are recommended to use NumPy since it results in more compact code, but feel free to use pure Python if you prefer.

If anything is unclear, do not hesitate to ask. Also, if you think some assumptions are missing, make your own and explain them!

Initialization

```
import numpy as np
import matplotlib.pyplot as plt
import sklearn # This is only to generate a dataset
```

Datasets

We start by making a synthetic dataset of 5000 instances and ten classes, with 500 instances in each class. (See https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_blobs.html regarding how the data are generated.) We choose to use a synthetic dataset---and not a set of natural occuring data---because we are mostly interested in properties of the various learning algorithms, in particular the differences between linear classifiers and multi-layer neural networks together with the difference between binary and multi-class data. In addition, we would like a dataset with instances represented with only two numerical features, so that it is easy to visualize the data. It would be rather difficult (although not impossible) to find a real-world dataset of the same nature. Anyway, you surely can use the code in this assignment for training machine learning models on real-world datasets.

When we are doing experiments in supervised learning, and the data are not already split into training and test sets, we should start by splitting the data. Sometimes there are natural ways to split the data, say training, on data from one year and testing on data from a later year, but if that is not the case, we should shuffle the data randomly before splitting. (OK, that is not necessary with this particular synthetic data set, since it is already shuffled by default by Scikit-Learn, but that will not be the case with real-world data) We should split the data so that we keep the alignment between X (features) and t (class labels), which may be achieved by shuffling the indices. We split into 60% for training, 20% for validation, and 20% for final testing. The set for final testing *must not be used* till the end of the assignment in part 3.

We fix the seed both for data set generation and for shuffling, so that we work on the same datasets when we rerun the experiments. This is done by the `random_state` argument and the `rng = np.random.RandomState(424242)`.

```
# Generating the dataset
from sklearn.datasets import make_blobs
X, t_multi = make_blobs(n_samples=[500, 500, 500, 500, 500, 500, 500,
500, 500, 500], centers=[[0,1],[4,2],[8,1],[2,0],[6,0],[3,-3],[4,-2],
[0,5],[0,4],[-2,-2]],
                  n_features=2, random_state=424242, cluster_std=[1.0,
2.0, 1.0, 0.5, 0.5, 3.0, 1.0, 0.5, 2.5, 2.5])

# Shuffling the dataset
indices = np.arange(X.shape[0])
rng = np.random.RandomState(424242)
rng.shuffle(indices)
indices[:10]
```

```
array([3560, 4674,   49, 1257,  661, 3066, 3834, 4792,  570, 3855])
```

```
# Splitting into train, dev and test
X_train = X[indices[:3000],:]
X_val = X[indices[3000:4000],:]
X_test = X[indices[4000:],:]
t_multi_train = t_multi[indices[:3000]]
t_multi_val = t_multi[indices[3000:4000]]
t_multi_test = t_multi[indices[4000:]]
```

Next, we will make a second dataset with only two classes by merging the existing labels in (X,t), so that `0-5` become the new `0` and `6-9` become the new `1`. Let's call the new set (X, t2). This will be a binary set. We now have two datasets:

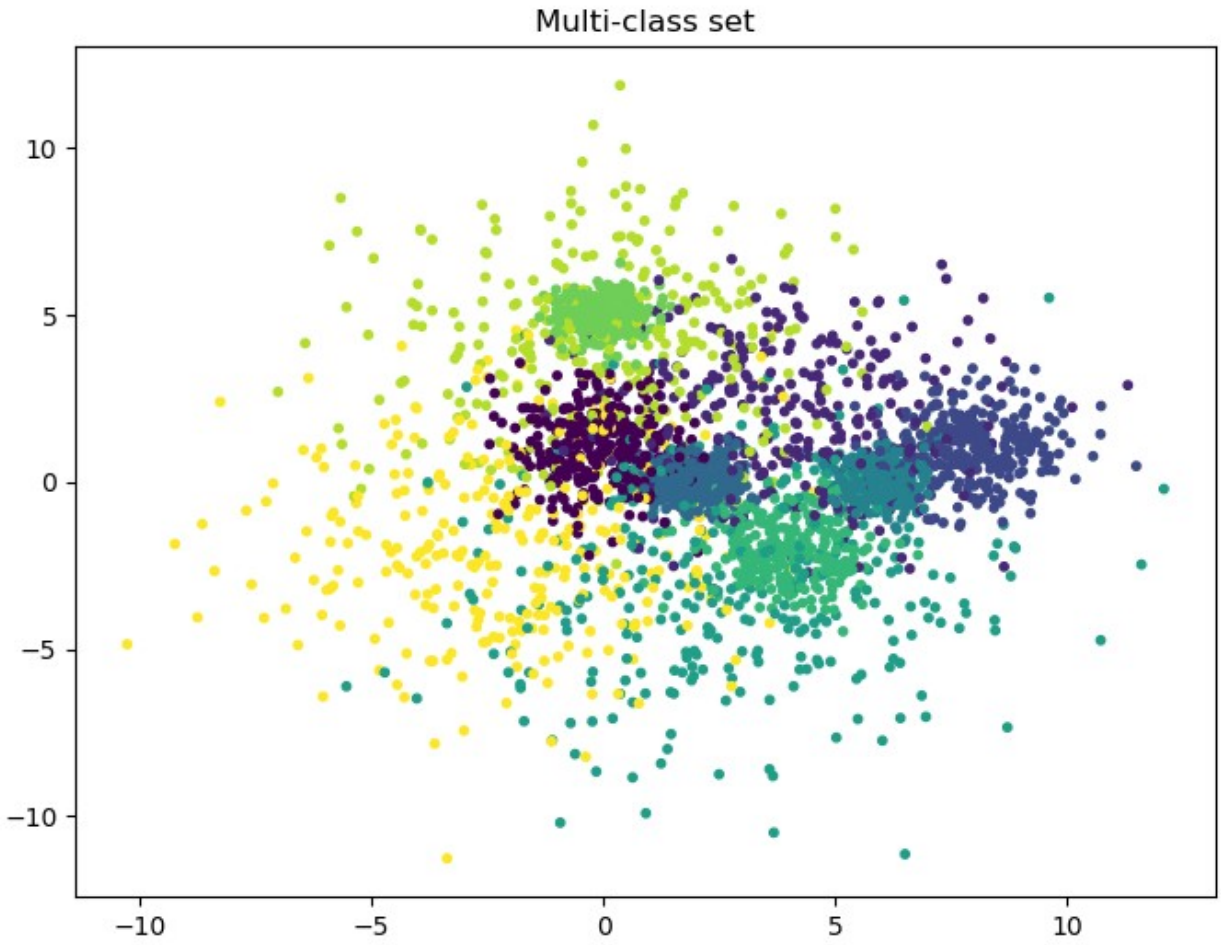
- Binary set: `(X, t2)`
- Multi-class set: `(X, t_multi)`

```
t2_train = t_multi_train >= 6
t2_train = t2_train.astype("int")
t2_val = (t_multi_val >= 6).astype("int")
t2_test = (t_multi_test >= 6).astype("int")
```

We can plot the two traning sets.

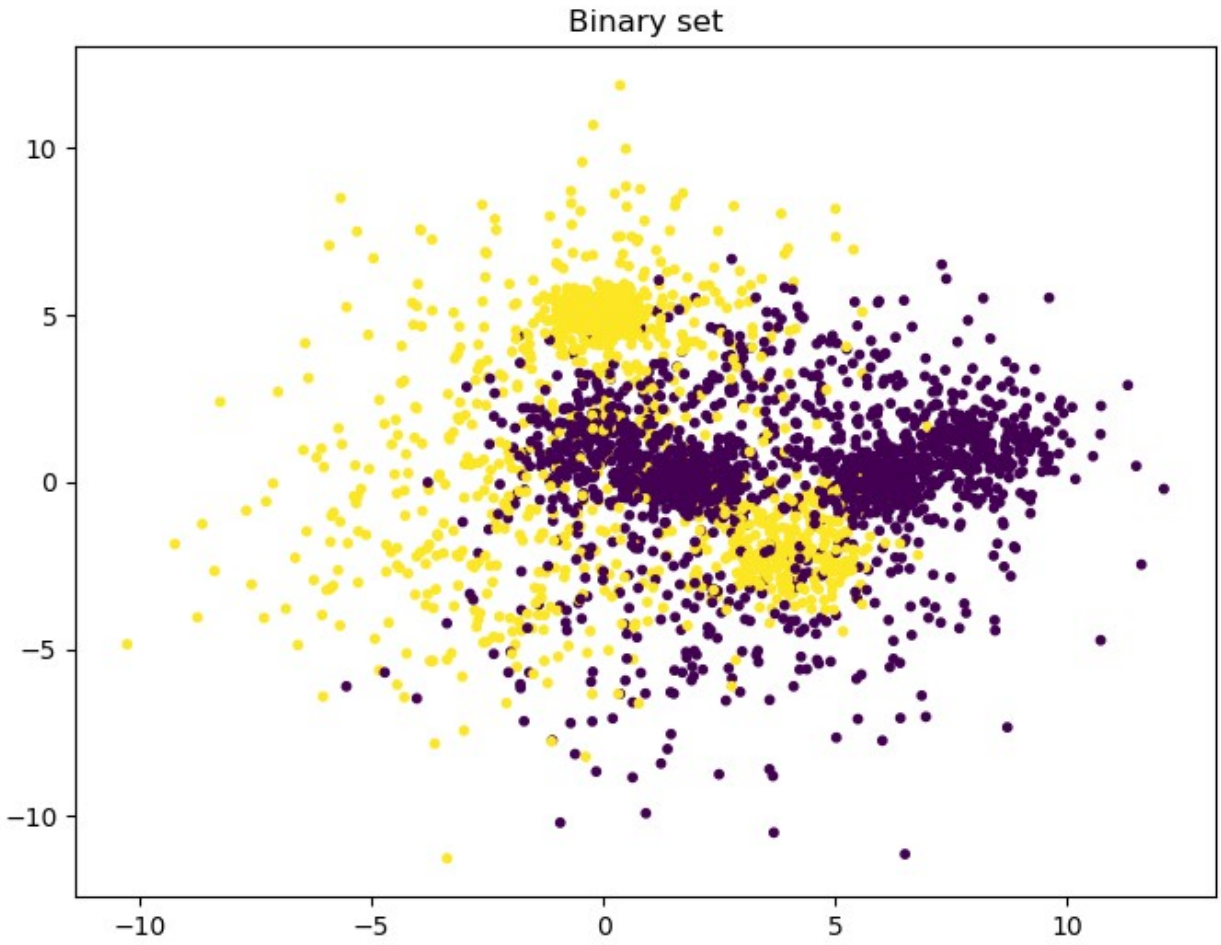
```
plt.figure(figsize=(8,6)) # You may adjust the size
plt.scatter(X_train[:, 0], X_train[:, 1], c=t_multi_train, s=10.0)
plt.title("Multi-class set")
```

```
Text(0.5, 1.0, 'Multi-class set')
```




```
plt.figure(figsize=(8,6))
plt.scatter(X_train[:, 0], X_train[:, 1], c=t2_train, s=10.0)
plt.title("Binary set")
```

```
Text(0.5, 1.0, 'Binary set')
```



Part 1: Linear classifiers

Linear regression

We see that even the binary set (X, t2) is far from linearly separable, and we will explore how various classifiers are able to handle this. We start with linear regression with the Mean Squared Error (MSE) loss, although it is not the most widely used approach for classification tasks: but we are interested. You may make your own implementation from scratch or start with the solution to the weekly exercise set 6. We include it here with a little added flexibility.

```
def add_bias(X, bias):
    """X is a NxM matrix: N datapoints, M features
    bias is a bias term, -1 or 1, or any other scalar. Use 0 for no
bias
    Return a Nx(M+1) matrix with added bias in position zero
    """
    N = X.shape[0]
    biases = np.ones((N, 1)) * bias # Make a N*1 matrix of biases
```

```
    # Concatenate the column of biases in front of the columns of X.
    return np.concatenate((biases, X), axis  = 1)

class NumpyClassifier():
    """Common methods to all Numpy classifiers --- if any"""

class NumpyLinRegClass(NumpyClassifier):

    def __init__(self, bias=-1):
        self.bias=bias

    def fit(self, X_train, t_train, lr = 0.1, epochs=10):
        """X_train is a NxM matrix, N data points, M features
        t_train is avector of length N,
        the target class values for the training data
        lr is our learning rate
        """

        if self.bias:
            X_train = add_bias(X_train, self.bias)

        (N, M) = X_train.shape

        self.weights = weights = np.zeros(M)

        for epoch in range(epochs):
            # print("Epoch", epoch)
            weights -= lr / N *  X_train.T @ (X_train @ weights -
t_train)

    def predict(self, X, threshold=0.5):
        """X is a KxM matrix for some K>=1
        predict the value for each point in X"""
        if self.bias:
            X = add_bias(X, self.bias)
        ys = X @ self.weights
        return ys > threshold
```

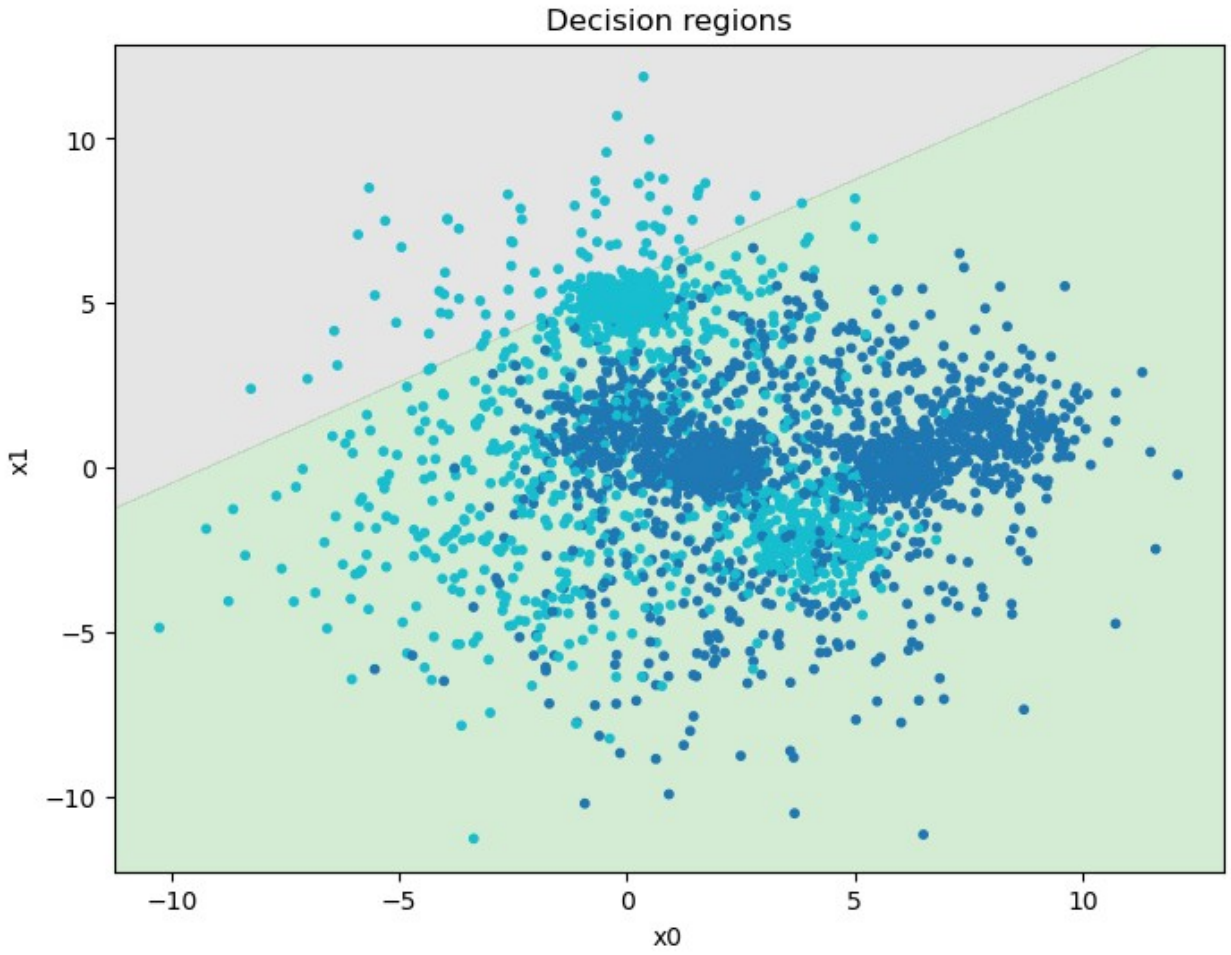
We can train and test a first classifier (on the binary dataset).

```
def accuracy(predicted, gold):
    return np.mean(predicted == gold)

cl = NumpyLinRegClass()
cl.fit(X_train, t2_train, lr=0.1, epochs=10)
accuracy(cl.predict(X_val), t2_val)
```

```
0.638
```

The following is a small procedure which plots the data set together with the decision boundaries. You may modify the colors and the rest of the graphics as you like. The procedure will also work for multi-class classifiers

```
def plot_decision_regions(X, t, clf=[], size=(8,6)):
    """Plot the data set (X,t) together with the decision boundary of
the classifier clf"""
    # The region of the plane to consider determined by X
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    # Make a prediction of the whole region
    h = 0.02  # step size in the mesh
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min,
y_max, h))
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    # Classify each meshpoint.
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=size) # You may adjust this

    # Put the result into a color plot
    plt.contourf(xx, yy, Z, alpha=0.2, cmap = 'tab10')

    plt.scatter(X[:,0], X[:,1], c=t, s=10.0, cmap='tab10')

    plt.xlim(xx.min(), xx.max())
    plt.ylim(yy.min(), yy.max())
    plt.title("Decision regions")
    plt.xlabel("x0")
    plt.ylabel("x1")
    plt.show()

plot_decision_regions(X_train, t2_train, cl)
```



Task: Tuning

The result is far from impressive. Remember that a classifier which always chooses the majority class will have an accuracy of 0.6 on this data set.

Your task is to try various settings for the two training hyper-parameters, learning rate and the number of epochs, to get the best accuracy on the validation set.

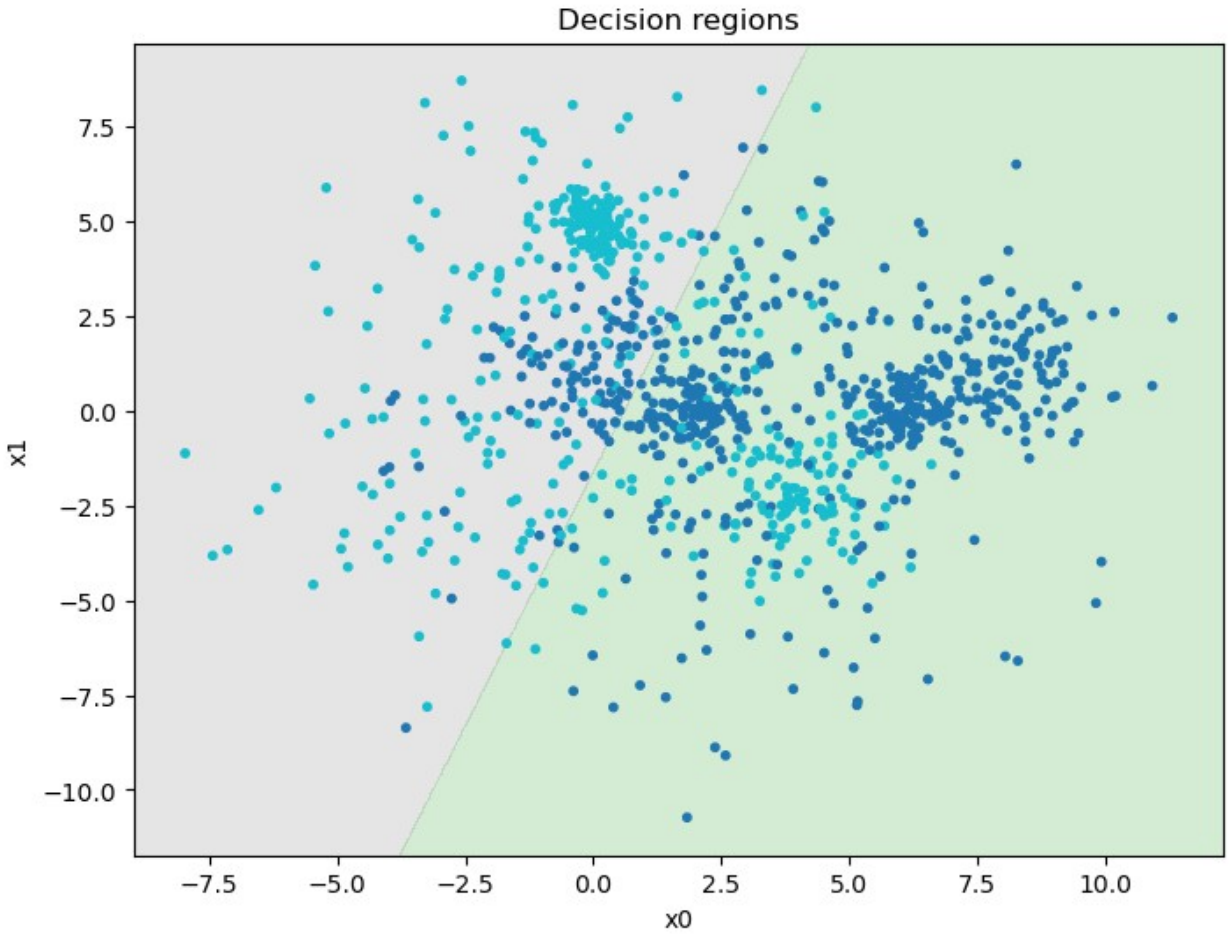
Report how the accuracy varies with the hyper-parameter settings. It it not sufficient to give the final hyperparameters. You must also show how you found then and results for alternative values you tried aout.

When you are satisfied with the result, you may plot the decision boundaries, as above.

```
# My settings
epochs = 10000         # Epochs
learning_rate = 0.01   # Learning rate

# Setup...
cl = NumpyLinRegClass()
```

```
# Fitting a line such that we minimize the MSE via a gradient descent
cl.fit(X_train, t2_train, lr=learning_rate, epochs=epochs)

# Getting accuracy
ac = accuracy(cl.predict(X_val), t2_val)

# Printing accuracy result
print(f"Accuracy: {ac}")

# Plotting result
plot_decision_regions(X_val, t2_val, cl)

# How I found the settings: More epochs and lower learning rate ->
More accurate results
# This is kind of how gradient descent works. The lower the learning
rate the more epochs are needed.
# It becomes slower, but that gives us less room for error (aka
precision rises).
# I simply increased number of epochs and decreased the learning rate
to be accurate and precise.

# I tried with 1000 epochs too, which is 0.2% less accurate than
10000.
# Sure, 10000 loops sounds like a lot, but it is relatively fast
either way.
# I personally didn't notice a difference between 1000 and 10000.

# That is basically it. I tried once with less epochs afterwards
# which did not go well, as was expected. Higher learning rate however
gave
# a runtime error since it could never surpass the accuracy threshold.
# The reason for that was the precision which was hilariously bad.

# At the end 10000 epochs and a learning rate 0.01 worked like a charm
# The end accuracy is 0.755 or 75.5%. Great job me :)
```

```
Accuracy: 0.755
```



Task: Scaling

We have seen in the lectures that scaling the data may improve training speed and sometimes the performance.

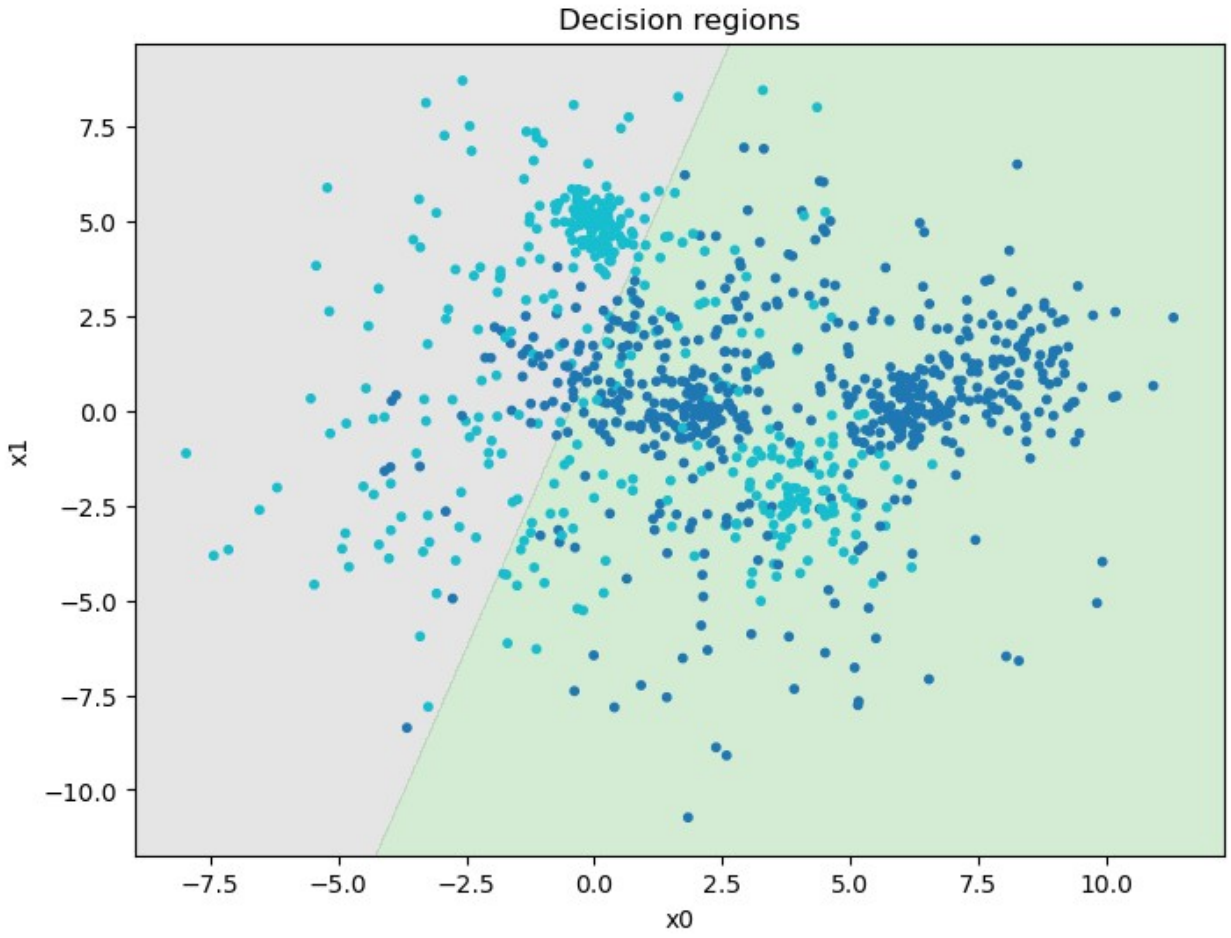
- Implement the standard scaler (normalizer); you can also try other scaling techniques
- Scale the data
- Train the model on the scaled data
- Experiment with hyper-parameter settings and see whether you can speed up or improve the training.
- Report final hyper-parameter settings and show how you found them.

```
# So here we will keep the settings from the previous task
# We will only experiment with the scale such that the
# accuracy increases above the previous 0.755 or 75.5%

# Scaling with the help of numpy
mean = X_train.mean(axis=0)
standard_deviation = X_train.std(axis=0)
X_train_scaled = (X_train - mean) / standard_deviation
```

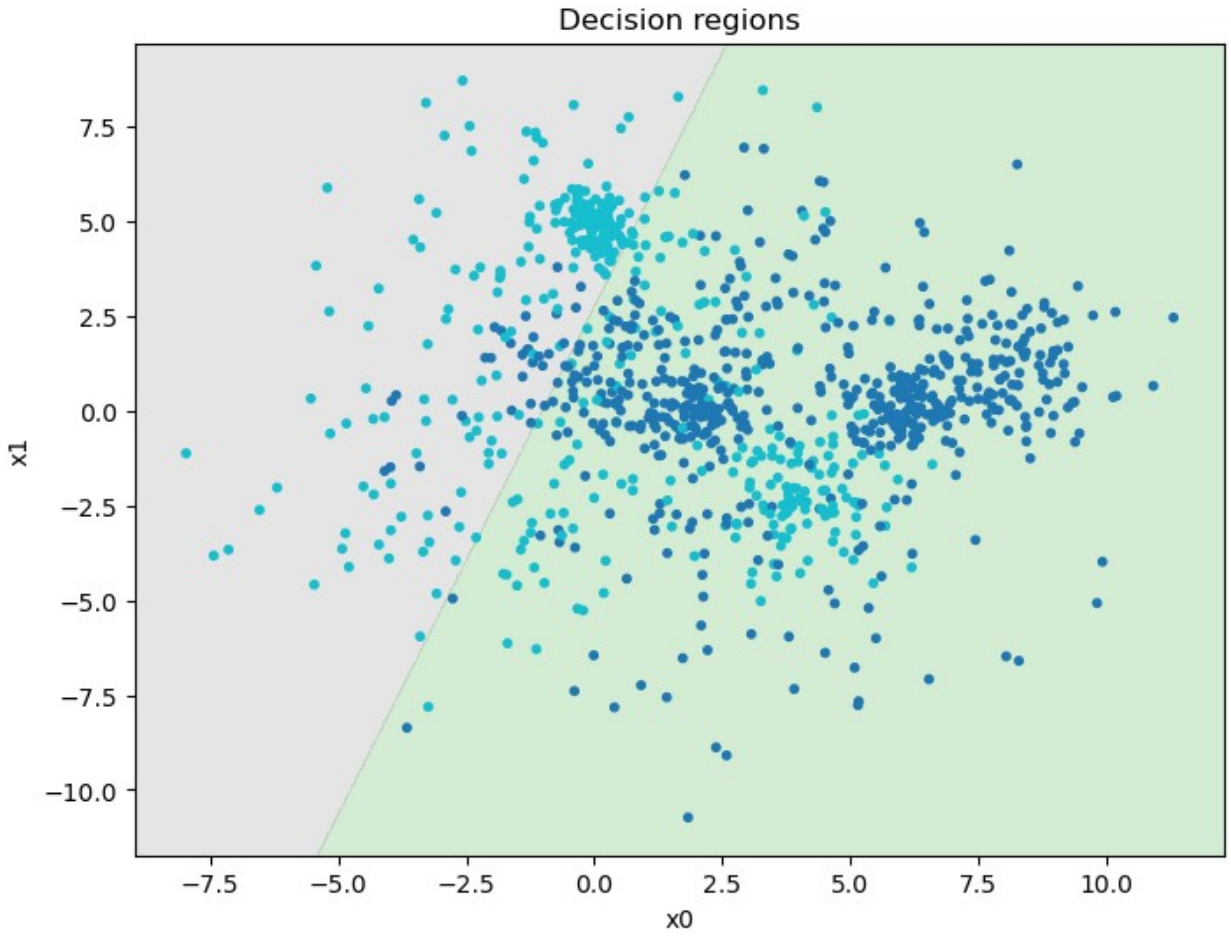
```
# Same steps as in previous task, just replaced X_train with
X_train_scaled
cl = NumpyLinRegClass()
cl.fit(X_train_scaled, t2_train, lr=learning_rate, epochs=epochs)
ac = accuracy(cl.predict(X_val), t2_val)

# Printing accuracy and plotting result
print(f"Accuracy: {ac}")
plot_decision_regions(X_val, t2_val, cl)
print()

# As you can see after standardizing the data points we got
# an accuracy of 0.765 or 76.5%. That is pretty neat!

# I also decided to brute force the size just to see what happens.
# By the way, this is what I got from it: 0.773 or 77.3% accuracy.
# My life is a lie ;(

run = True
if run:

    # Some settings
    epochs = 1000
    learning_rate = 0.1
    test_range = (-4, 0)
    step = 0.1

    # Finding best shift in data
    best = {"accuracy": 0, "offset": 0, "cl": None}
    for n in range(round((test_range[1] - test_range[0]) / step)):

        # Trying out next offset
        offset = round(n * step + test_range[0], 4)
        X_train_scaled = X_train + offset

        # Same steps as in previous task, just replaced X_train with
X_train_scaled
        cl = NumpyLinRegClass()
        cl.fit(X_train_scaled, t2_train, lr=learning_rate,
epochs=epochs)
        ac = accuracy(cl.predict(X_val), t2_val)

        # Checking if this is better than previous best
        if ac > best["accuracy"]:
            best["accuracy"] = ac
            best["offset"] = offset
            best["cl"] = cl

    # Printing and plotting result
    print(f"Accuracy: {best["accuracy"]}")
```

```
    print(f"Offset: {best["offset"]}")
    plot_decision_regions(X_val, t2_val, best["cl"])

# My theory: Due to the way MSE works the weights are getting
# punished more severely based on y distance. -2.5 just so happens
# to be the offset to the data which counteracts this nature the best.
# For the same reason division and multiplication helps as well.

# So basically, MSE is a wrong choice here. MSE doesn't care about
# whether the classification is correct or wrong, only the error
distance.
# So even though the classification is either wrong or correct, if
there
# are few wrong, but the distance is far greater the MSE perceives is
at more
# wrong than if there are slightly more wrongly classified with small
distance.

# Since it is MSE's gradient descent that adjusts the weights, aka the
distance
# and not the correct/wrong classification, we can't use it to predict
accurately.
# It is sort of an approximation, but more extreme cases are
definitely possible.

# This is the core of the problem and the only reason why the
perceptron can't
# get more accurate. By shifting and resizing the data points we
accidentally fix
# this problem with the error distance. Fortunately in our case it
can't get worse.
```

```
Accuracy: 0.765
```




```
Accuracy: 0.773
Offset: -2.6
```



Logistic regression

a) You should now implement a logistic regression classifier similarly to the classifier based on linear regression. You may use the code from the solution to weekly exercise set week06.

b) In addition to the method `predict()` which predicts a class for the data, include a method `predict_probabilities()` which predicts the probability of the data belonging to the positive class.

c) So far, we have not calculated the loss explicitly in the code. Extend the code to calculate the loss on the training set for each epoch and to store the losses such that the losses can be inspected after training. The prefered loss for logistic regression is binary cross-entropy, but you can also try mean squared error. The most important is that your implementation of the loss corresponds to your implementation of the gradient descent. Also, calculate and store accuracies after each epoch.

d) In addition, extend the `fit()` method with optional arguments for a validation set (X_val, t_val). If a validation set is included in the call to `fit()`, calculate the loss and the accuracy for the validation set after each epoch.

e) The training runs for a number of epochs. We cannot know beforehand for how many epochs it is reasonable to run the training. One possibility is to run the training until the learning does

not improve much. Extend the `fit()` method with two keyword arguments, `tol` (tolerance) and `n_epochs_no_update` and stop training when the loss has not improved with more than `tol` after `n_epochs_no_update` (to save compute and potentially avoid over-fitting). A possible default value for `n_epochs_no_update` is 2. Also, add an attribute to the classifier which tells us after fitting how many epochs it was trained for.

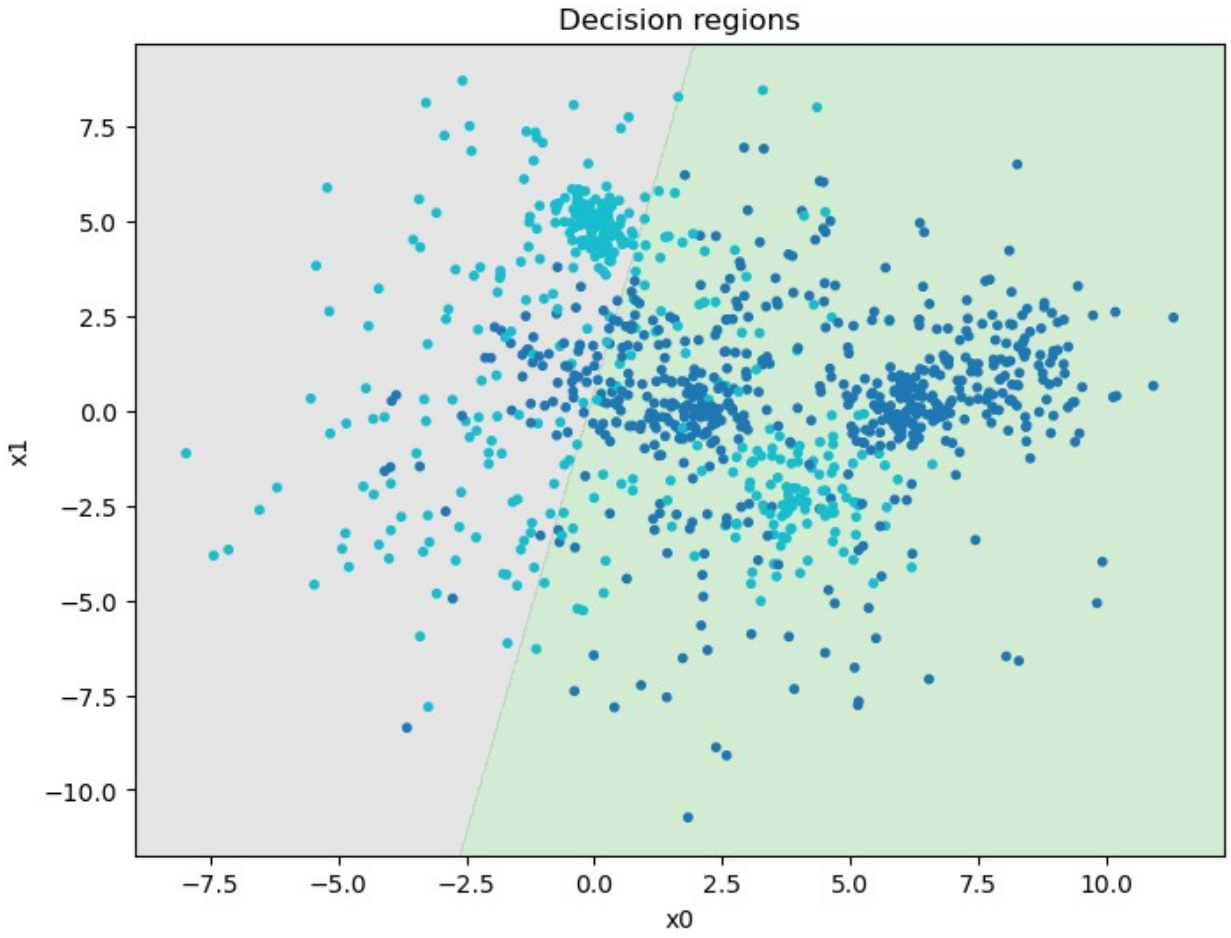
f) Train classifiers with various learning rates, and with varying values for `tol` for finding the optimal values. Also consider the effect of scaling the data.

g) After a succesful training, for your best model, plot both training loss and validation loss as functions of the number of epochs in one figure, and both training and validation accuracies as functions of the number of epochs in another figure. Comment on what you see. Are the curves monotone? Is this as expected?

```
# Actually a copy paste of NumpyLinRegClass + some changes
class NumpyLogRegClass(NumpyLinRegClass):

    def __init__(self, bias=-1):
        super().__init__(bias)

    def fit(self, X_train, t_train, lr = 0.1, epochs=10):
        """X_train is a NxM matrix, N data points, M features
        t_train is avector of length N,
        the target class values for the training data
        lr is our learning rate
        """

        if self.bias:
            X_train = add_bias(X_train, self.bias)

        (N, M) = X_train.shape

        self.weights = np.zeros(M)

        for epoch in range(epochs):
            # print("Epoch", epoch)
            y = 1 / (1 + np.e ** -(X_train @ self.weights))     #
Sigmoid function from lectures
            self.weights -= lr / N * X_train.T @ (t_train - y)  #
Gradient descent from lectures

    def predict(self, X, threshold=0):
        """X is a KxM matrix for some K>=1
        predict the value for each point in X"""
        if self.bias:
            X = add_bias(X, self.bias)
        ys = X @ self.weights
        return ys < threshold  # We flip that for it to make sense

        # Slight change on return, because of the sigmoid function
```

```
        # Note that I also changed threshold to 0, this is because
        # we now have a range of -infinity to +infinity.

        # We can convert this back to probabilities with sigmoid,
        # then we could go back to 0.5 as done previously.
        # Ultimately it isn't that important.

# My settings. Duplicate but flexible
epochs = 10000
learning_rate = 0.01

# Now with NumpyLogRegClass
cl = NumpyLogRegClass()
cl.fit(X_train, t2_train, lr=learning_rate, epochs=epochs)
ac = accuracy(cl.predict(X_val), t2_val)

# Printing accuracy and plotting result
print(f"Accuracy: {ac}")
plot_decision_regions(X_val, t2_val, cl)

# By the way, ignore the overflow error
```

```
C:\Users\Justa\AppData\Local\Temp\ipykernel_29000\55995779.py:23:
RuntimeWarning: overflow encountered in power
  y = 1 / (1 + np.e ** -(X_train @ self.weights))     # Sigmoid
function from lectures

Accuracy: 0.771
```



Multi-class classifiers

We turn to the task of classifying when there are more than two classes, and the task is to ascribe one class to each input. We will now use the set `(X, t_multi)`.

Multi-class with logistic regression

We saw in the lectures how a logistic regression classifier can be turned into a multi-class classifier using the one-vs-rest approach. We train one logistic regression classifier for each class. To predict the class of an item, we run all the binary classifiers and collect the probability score from each of them. We assign the class which ascribes the highest probability.

Build such a classifier. Train the resulting classifier on `(X_train, t_multi_train)`, test it on `(X_val, t_multi_val)`, tune the hyper-parameters and report the accuracy.

Also plot the decision boundaries for your best classifier similarly to the plots for the binary case.

For IN4050 students: Multinomial logistic regression

The following part is only mandatory for IN4050 students. IN3050 students are also welcome to make it a try. Everybody has to do the part 2 on multi-layer neural networks.

In the lectures, we contrasted the one-vs-rest approach with the multinomial logistic regression, also called softmax classifier. Implement also this classifier, tune the parameters, and compare the results to the classifiers above. (Don't expect a large difference on a simple task like this.)

Remember that this classifier uses softmax in the forward phase. For loss, it uses categorical cross-entropy loss. The loss has a somewhat simpler form than in the binary case. To calculate the gradient is a little more complicated. The actual gradient and update rule is simple, however, as long as you have calculated the forward values correctly.

Our classifier is not ideal. But is it because all the 10 classes are equally diffucult to predict? You should evaluate the multinomial model predictions for each class separately and report your findings. Please also report whether there is any difference in this respect between the training and the validation sets.

```
# First I want to reformat the t_train_multi.
# As explained in the lectures we must convert values into a binary
tuple
# So 2 for example would be (0, 0, 1, 0, 0, 0) for 6 labeled multi
train
def enformat_multi(t_multi):

    # Coppying for flexibility
    t_multi_reformated = t_multi.copy().tolist()

    # This is the amount of labels that we got
    labels_count = max(t_multi_reformated) + 1 -
min(t_multi_reformated)

    # Converting to the form we desire
    for index in range(len(t_multi_reformated)):

        # Creating an array with all possible labels
        labels = np.zeros(labels_count)

        # Assigning value to the correct label
        labels[t_multi_reformated[index]] = 1

        # Replacing the previous format
        t_multi_reformated[index] = labels

    # Converting back to numpy array and returning
    t_multi_reformated = np.asarray(t_multi_reformated)
    return t_multi_reformated

# I also want a way to get it back to the way it was formated first
# However I never really used it, I was simply saving the original
def deformat_multi(t_multi_reformated):

    # Coppying for flexibility
```

```
    t_multi = t_multi_reformated.copy().tolist()
    labels_count = len(t_multi[0])

    # Converting to the form we desire
    for index in range(len(t_multi)):

        # Finding the value
        for num in range(labels_count):
            if t_multi[index][num] == 1:

                # Reverting back to the original
                t_multi[index] = num  # We assume that we started with
0
                break

    # Converting back to numpy array and returning
    t_multi = np.asarray(t_multi)
    return t_multi

# Example of reformating
t_multi_train_new = enformat_multi(t_multi_train)
print(t_multi_train_new)

# Space for clarity
print()

# Example of reformating it back to the original
t_multi_train_new = deformat_multi(t_multi_train_new)
print(t_multi_train_new)
```

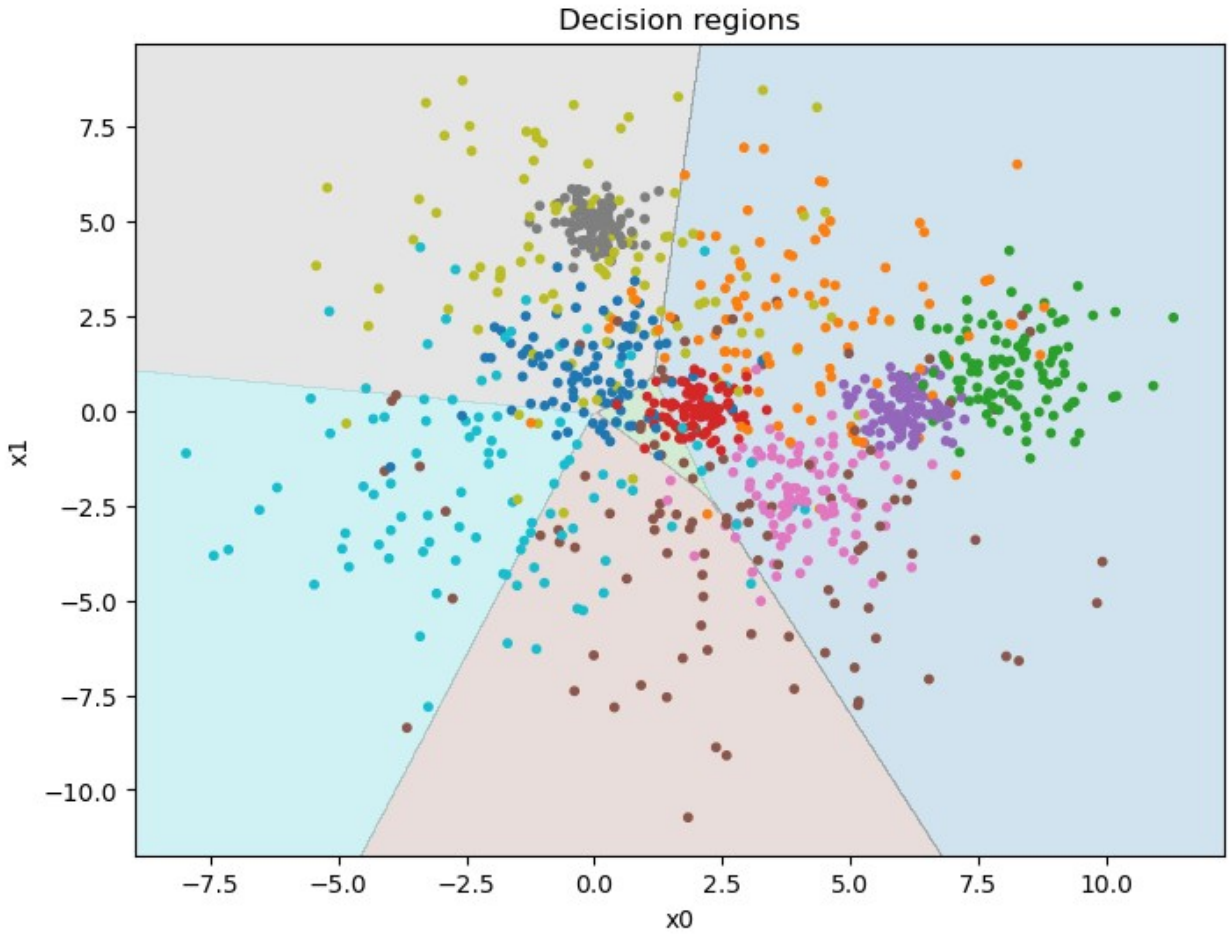
```
[[0. 1. 0. ... 0. 0. 0.]
 [0. 0. 1. ... 0. 0. 0.]
 [0. 0. 1. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [1. 0. 0. ... 0. 0. 0.]
 [0. 0. 1. ... 0. 0. 0.]]

[1 2 2 ... 3 0 2]
```

```
class NumpyLogRegMultiClass(NumpyLogRegClass):

    def __init__(self, bias=-1):
        super().__init__(bias)

    def fit(self, X_train, t_train, lr = 0.1, epochs=10):

        # We simply predict each one separately now as shown in the
lectures
        self.multi_weights = []  # Here we will store the weights that
```

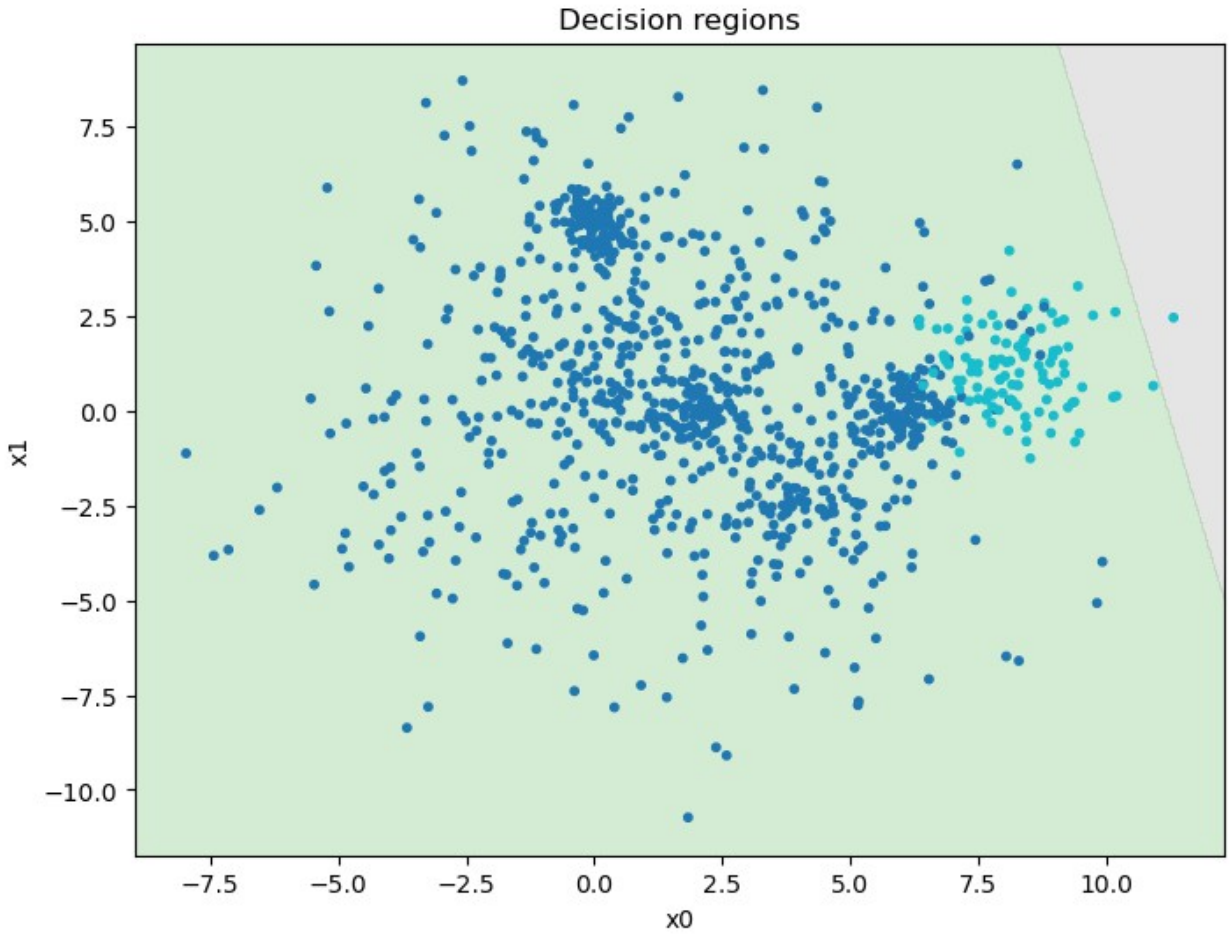
```
we get
        labels = len(t_train[0])
        for label in range(labels):

            # Now we basically repeat the process for each label
            super().fit(X_train, t_train[:, label], lr, epochs)

            # We save the weights that we got into our new list
            self.multi_weights.append(self.weights.copy())
        self.multi_weights = np.asarray(self.multi_weights)

    def predict(self, X, threshold=0):

        # Copy paste
        """X is a KxM matrix for some K>=1
        predict the value for each point in X"""
        if self.bias:
            X = add_bias(X, self.bias)

        # To predict we do the exact same thing,
        # but we compare with all the weights and choose
        # the one with the highest certainty (value)

        # Unfortunately I have no idea how to graph it
        # If I will have enough time and willpower I will
        # implement a solution, but probably not...

        # We assume that labels are assigned numerically
        # We also assume that labels start with 0

        # Assigning labels based on the one with the highest certainty
        ys = []
        for row in X:

            # Best so far
            best_label = 0
            ys_i = row @ self.multi_weights[best_label]
            best_certainty = 1 / (1 + np.e ** -ys_i)

            # Comparing certainty with other specialized weights
            for weights in range(1, len(self.multi_weights)):
                ys_i = row @ self.multi_weights[weights]
                certainty = 1 / (1 + np.e ** -ys_i)

                # Comparing certainty of a weight A with the certainty
of a weight B
                if certainty < best_certainty:  # We take the least
one because of the {ys < threshold} thing

                    # Override
```

```
                    best_label = weights
                    best_certainty = certainty

            # Adding to ys list
            ys.append(best_label)

        # Returning prediction
        ys = np.asarray(ys)
        return ys

# My settings. Duplicate but flexible
epochs = 100
learning_rate = 0.1

# Don't forget to reformat for training
t_multi_train_reformated = enformat_multi(t_multi_train)

# Now with NumpyLogRegClass
cl = NumpyLogRegMultiClass()
cl.fit(X_train, t_multi_train_reformated, lr=learning_rate,
epochs=epochs)
ac = accuracy(cl.predict(X_val), t_multi_val)

# Printing accuracy and plotting result
print(f"Accuracy: {ac}")
plot_decision_regions(X_val, t_multi_val, cl)

# First of all I am surprised by how long it took to draw.
# But I did not check the code for plotting so... yeah.

# Also, if you want try increasing number of epochs and decreasing the
learning rate yourself
# Apparently there is a weight which always takes over.
# The reason for that is that it is the first I choose as the best,
and more epochs
# allow for the rest of the weights to come closer to the 0% certainty
which they
# all share at the end (except for 5, 7, and 9 which work properly,
thank God)
# 5, 7 and 9 are brown, gray, and cyan

# As you can see it does not work... to its fullest.
# There are lots of logistical regressions that simply were bad at
their job.
# I have tried for a whole day straight to fix this just to realize
one thing.
# It, apparently, has multiple local minima. Or so I believe.
# This means that I might get stuck on the local minimum and never
reach the global minimum.
# Here is an example I discovered for which increasing epochs and
```

```
decreasing the learning rate did not help.

# Note the way we split the data
t_custom_train = t_multi_train == 2
t_custom_train = t_custom_train.astype("int")
t_custom_val = t_multi_val == 2
t_custom_val = t_custom_val.astype("int")

# My default settings
epochs = 10000
learning_rate = 0.01

# Note that I did not change the class
cl = NumpyLinRegClass()
cl.fit(X_train, t_custom_train, lr=learning_rate, epochs=epochs)

# Now we predict and print, observe
ac = accuracy(cl.predict(X_val), t_custom_val)
print(f"Accuracy: {ac}")
plot_decision_regions(X_val, t_custom_val, cl)

# See how it got stuck at such a weird spot?
# You can obviously see that there is a way better one.
# How did that even happen? Is YOUR code wrong?
# I don't think so... It is not wrong (I triple checked).
# To be honest I am not even sure if that is a local minimum anymore.
# And if it is, I thought we weren't supposed to have any here.
# Please explain ;(

Accuracy: 0.189
```




```
Accuracy: 0.893
```



Part 2 Multi-layer neural networks

A first non-linear classifier

The following code is a simple implementation of a multi-layer perceptron or feed-forward neural network. For now, it is quite restricted. There is only one hidden layer with 6 neurons. It can only handle binary classification. In addition, it uses a simple final layer similar to the linear regression classifier above. One way to look at it is what happens when we add a hidden layer to the linear regression classifier.

The MLP class below misses the implementation of the `forward()` function. Your first task is to implement it.

Remember that in the forward pass, we "feed" the input to the model, the model processes it and produces the output. The function should make use of the logistic activation function and bias.

```
# First, we define the logistic function and its derivative:
def logistic(x):
```

```
    return 1/(1+np.exp(-x))

def logistic_diff(y):
    return y * (1 - y)

class MLPBinaryLinRegClass(NumpyClassifier):
    """A multi-layer neural network with one hidden layer"""

    def __init__(self, bias=-1, dim_hidden = 6):
        """Intialize the hyperparameters"""
        self.bias = bias
        # Dimensionality of the hidden layer
        self.dim_hidden = dim_hidden

        self.activ = logistic

        self.activ_diff = logistic_diff

        # Accuracy and loss declarations
        self.accuracy = []
        self.loss = []

    def forward(self, X):
        """TODO:
        Perform one forward step.
        Return a pair consisting of the outputs of the hidden_layer
        and the outputs on the final layer"""

        # Doing some calculations
        hidden = self.activ(X @ self.weights1)             # Applying
first weights (aka hidden layer)
        hidden_outs = add_bias(hidden, self.bias)          # Adding
bias
        outputs = self.activ(hidden_outs @ self.weights2)  # And now
the second layer

        # Returning the hidden outputs and the outputs
        return hidden_outs, outputs

    def fit(self, X_train, t_train, X_val=None, t_val=None, lr=0.001,
epochs=100, tol=0.01, n_epochs_no_update=2, ):
        """Initialize the weights. Train *epochs* many epochs.

        X_train is a NxM matrix, N data points, M features
        t_train is a vector of length N of targets values for the
training data,
        where the values are 0 or 1.
        lr is the learning rate
        """
        self.lr = lr
```

```
        # Turn t_train into a column vector, a N*1 matrix:
        T_train = t_train.reshape(-1,1)

        dim_in = X_train.shape[1]
        dim_out = T_train.shape[1]

        # Initialize the weights
        self.weights1 = (np.random.rand(
            dim_in + 1,
            self.dim_hidden) * 2 - 1)/np.sqrt(dim_in)
        self.weights2 = (np.random.rand(
            self.dim_hidden+1,
            dim_out) * 2 - 1)/np.sqrt(self.dim_hidden)
        X_train_bias = add_bias(X_train, self.bias)

        # No updates counter
        no_updates_counter = n_epochs_no_update

        for e in range(epochs):
            # One epoch
            # The forward step:
            hidden_outs, outputs = self.forward(X_train_bias)
            # The delta term on the output node:
            out_deltas = (outputs - T_train)
            # The delta terms at the output of the hidden layer:
            hiddenout_diffs = out_deltas @ self.weights2.T
            # The deltas at the input to the hidden layer:
            hiddenact_deltas = (hiddenout_diffs[:, 1:] *
                                self.activ_diff(hidden_outs[:, 1:]))

            # Update the weights:
            self.weights2 -= self.lr * hidden_outs.T @ out_deltas
            self.weights1 -= self.lr * X_train_bias.T @
hiddenact_deltas

            # Accuracy and loss update
            try:
                X_val.shape
                t_val.shape
            except AttributeError:
                pass
            else:

                # Updating accuracy
                ac = accuracy(self.predict(X_val), t_val)
                self.accuracy.append(ac)

                # Updating loss
```

```
                predicted_probability =
self.predict_probabilities(X_val)
                ls = -(t_val * np.log(predicted_probability) + (1 -
t_val) * np.log(1 - predicted_probability))
                self.loss.append(ls.mean())

                # Checking if improved enough
                if len(self.loss) > 1:
                    if self.loss[-2] - self.loss[-1] < tol:
                        no_updates_counter -= 1
                    else:
                        no_updates_counter = n_epochs_no_update  #
Reset

            # Checking if we should stop
            if no_updates_counter < 1:
                break

    def predict(self, X):
        """Predict the class for the members of X"""
        Z = add_bias(X, self.bias)
        forw = self.forward(Z)[1]
        score= forw[:, 0]
        return (score > 0.5)

    # Predicting but with % and such
    # Literally a copy-paste
    def predict_probabilities(self, X):
        """Predict the class for the members of X"""
        Z = add_bias(X, self.bias)
        forw = self.forward(Z)[1]
        score = forw[:, 0]
        return score  # The only change
```

When implemented, this model can be used to make a non-linear classifier for the set `(X, t2)`. Experiment with settings for learning rate and epochs and see how good results you can get. Report results for various settings. Be prepared to train for a long time (but you can control it via the number of epochs and hidden size).

Plot the training set together with the decision regions as in Part I.

Improving the MLP classifier

You should now make changes to the classifier similarly to what you did with the logistic regression classifier in part 1.

a) In addition to the `predict()` method, which predicts a class for the data, include the `predict_probabilities()` method which predict the probability of the data belonging to the positive class. The training should be based on these values, as with logistic regression.

b) Calculate the loss and the accuracy after each epoch and store them for inspection after training.

c) Extend the `fit()` method with optional arguments for a validation set `(X_val, t_val)`. If a validation set is included in the call to `fit()`, calculate the loss and the accuracy for the validation set after each epoch.

d) Extend the `fit()` method with two keyword arguments, `tol` (tolerance) and `n_epochs_no_update` and stop training when the loss has not improved for more than `tol` after `n_epochs_no_update`. A possible default value for `n_epochs_no_update` is 2. Add an attribute to the classifier which tells us after fitting how many epochs it was trained on.

e) Tune the hyper-parameters: `lr`, `tol` and `dim-hidden` (size of the hidden layer). Also, consider the effect of scaling the data.

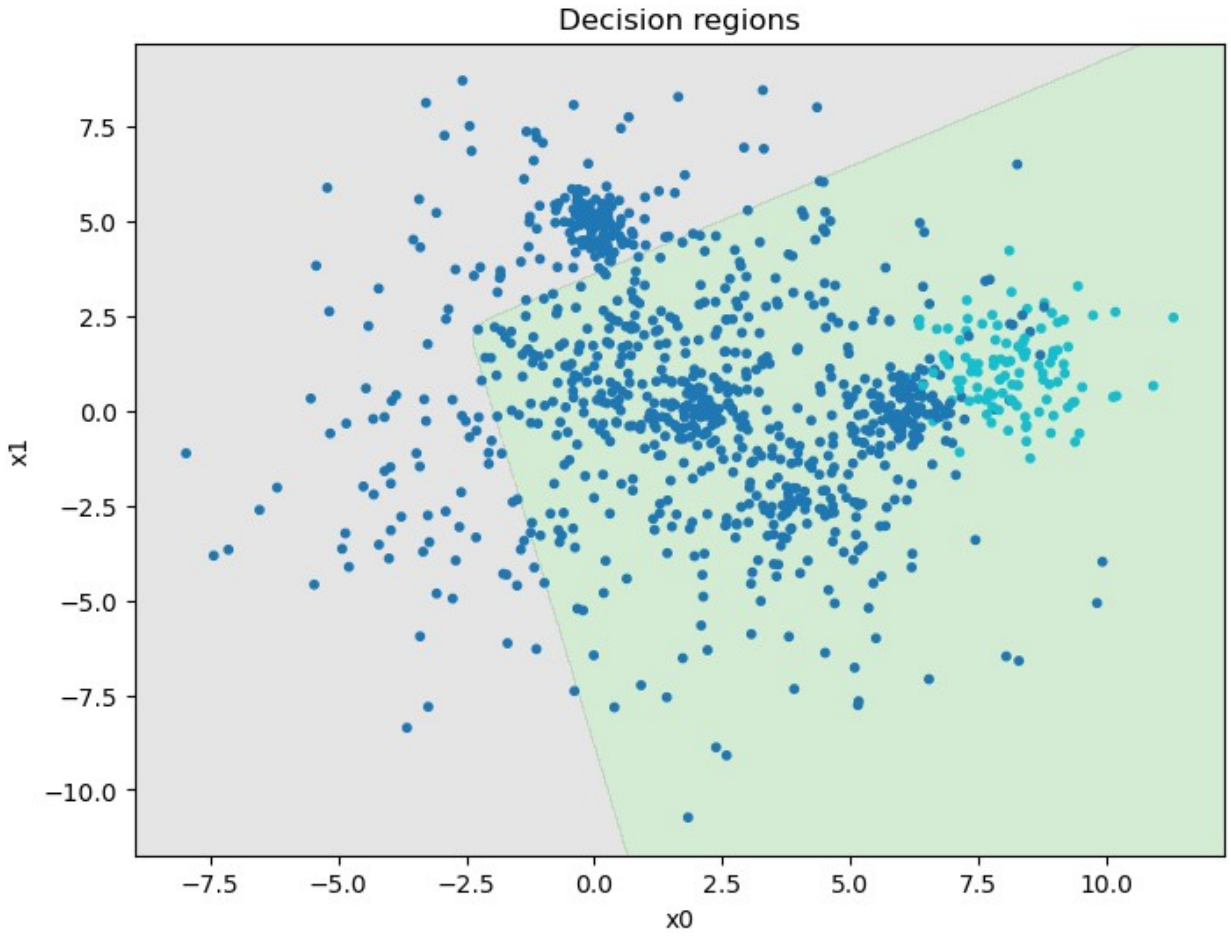
f) After a succesful training with the best setting for the hyper-parameters, plot both training loss and validation loss as functions of the number of epochs in one figure, and both training and validation accuracies as functions of the number of epochs in another figure. Comment on what you see.

g) The MLP algorithm contains an element of non-determinism. Hence, train the classifier 3 times with the optimal hyper-parameters and report the mean and standard deviation of the accuracies over the 3 runs.

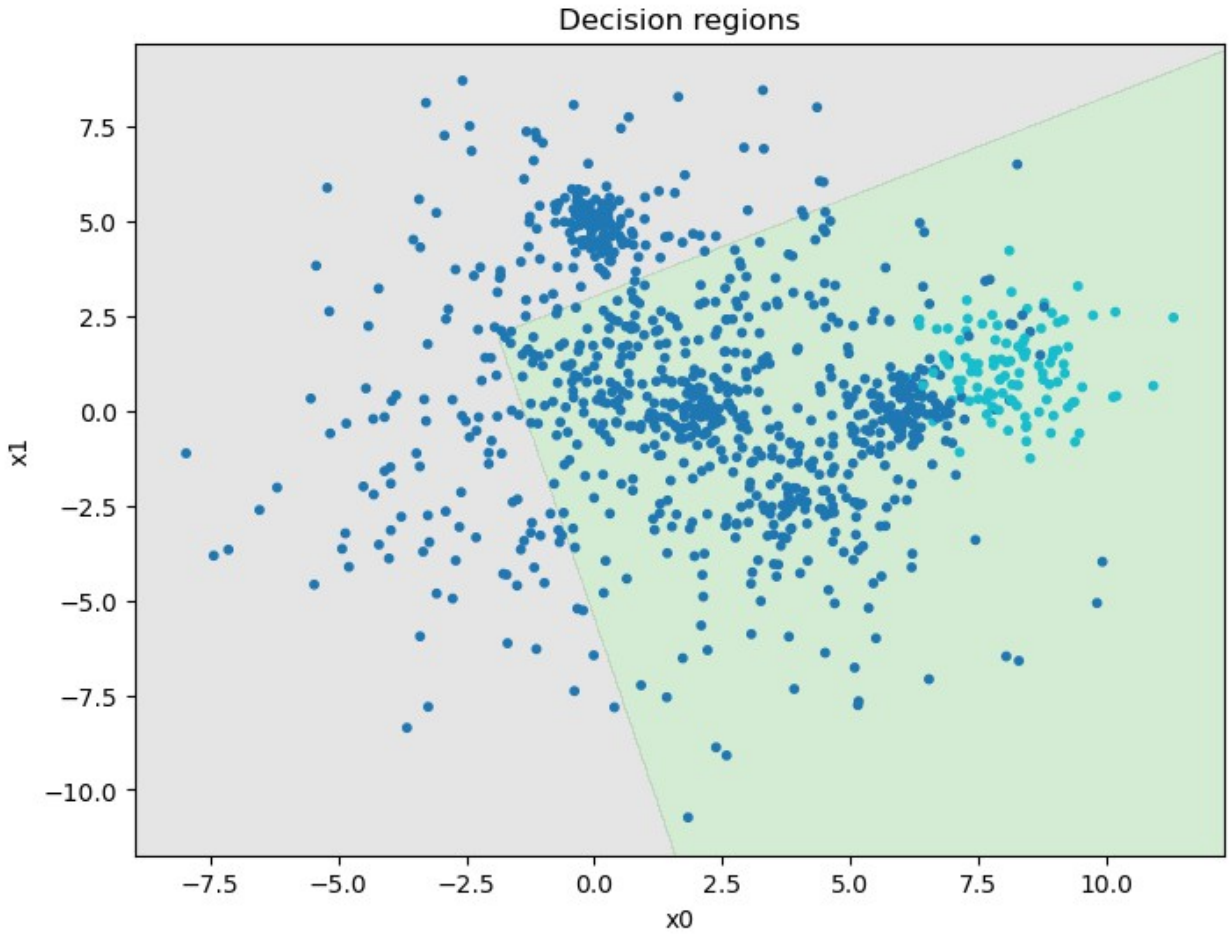
```
# My settings
dim_hidden = 2
epochs = 1000
learning_rate = 0.01
tolerance = 0.001
number_of_epochs_without_update = 6

# Some declarations
accuracies = []

# Repeating the process 3 times
runs = 3
for run in range(runs):

    # Training a model
    cl = MLPBinaryLinRegClass(dim_hidden=dim_hidden)
    cl.fit(X_train, t2_train, X_val, t2_val, lr=learning_rate,
epochs=epochs, tol=tolerance,
n_epochs_no_update=number_of_epochs_without_update)
    ac = accuracy(cl.predict(X_val), t2_val)  # Getting accuracy of a
prediction

    # Storing data
```

```
    accuracies.append(ac)

    # Printing and plotting
    print(f"Accuracy: {ac}")
    plot_decision_regions(X_val, t_custom_val, cl)

# Caluclating mean
mean = 0
for acc in accuracies:
    mean += acc
mean /= len(accuracies)

# Caluclating standard deviation
standard_deviation = 0
for acc in accuracies:
    standard_deviation += (acc - mean) ** 2
standard_deviation = (standard_deviation / (len(accuracies) - 1)) **
0.5

# Printing results
print(f"Mean: {round(mean, 4)}")
print(f"Standard Deviation: {round(standard_deviation, 4)}")

# I honestly have no idea why everything is going so bad today.
# I don't know what I did wrong or if there is something I did not
implement.
# Forward seems to work. That is actually all that was needed.
# Perhaps I simply should not expect such great results from it,
# but the fact that it is barely over 60% accurate is really, not
good.
```

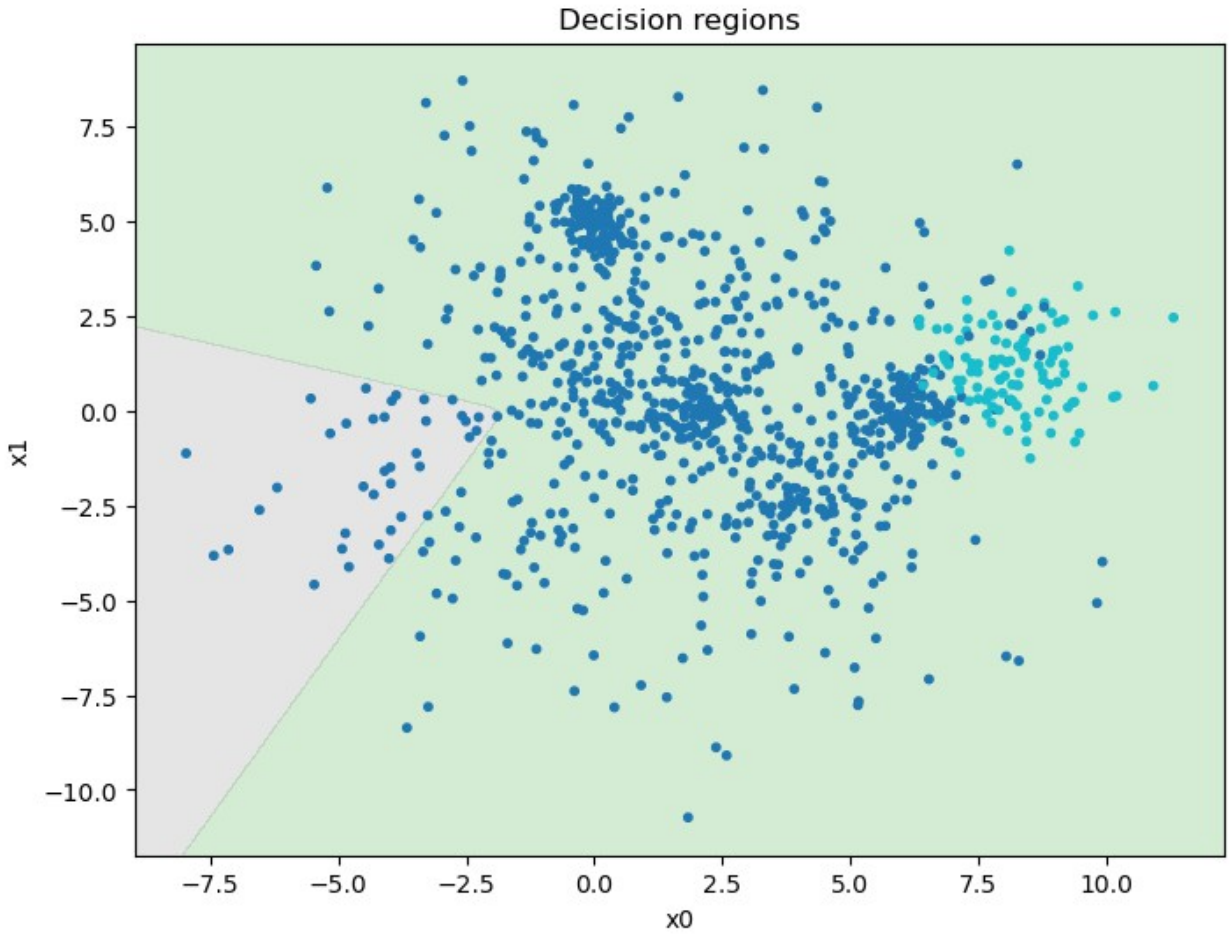
```
Accuracy: 0.799
```




```
Accuracy: 0.802
```




```
Accuracy: 0.623
```




```
Mean: 0.7413
Standard Deviation: 0.1025
```

For IN4050-students: Multi-class neural network

The following part is only mandatory for IN4050 students. IN3050 students are also welcome to make it a try. This is the most fun part of the set :))

The goal is to use a feed-forward neural network for non-linear multi-class classfication and apply it to the set `(X, t_multi)`.

Modify the network to become a multi-class multinomial classifier. As a sanity check of your implementation, you may apply it to `(X, t_2)` and see whether you get similar results as above.

Train the resulting classifier on `(X_train, t_multi_train)`, test it on `(X_val, t_multi_val)`, tune the hyper-parameters and report the accuracy.

Plot the decision boundaries for your best classifier. Evaluate the best model predictions for each class separately (same as in part 1) and report your findings. Please also report whether there is any difference in this respect between the training and the validation sets.

Part III: Final testing

We can now perform a final testing on the held-out test set we created in the beginning.

Binary task (X, t2)

Consider the linear regression classifier, the logistic regression classifier and the multi-layer network with the best settings you found. Train each of them on the training set and evaluate on the held-out test set, but also on the validation set and the training set. Report the performance in a 3 by 3 table.

Comment on what you see. How do the three different algorithms compare? Also, compare the results between the different dataset splits. In cases like these, one might expect slightly inferior results on the held-out test data compared to the validation and training data. Is this the case?

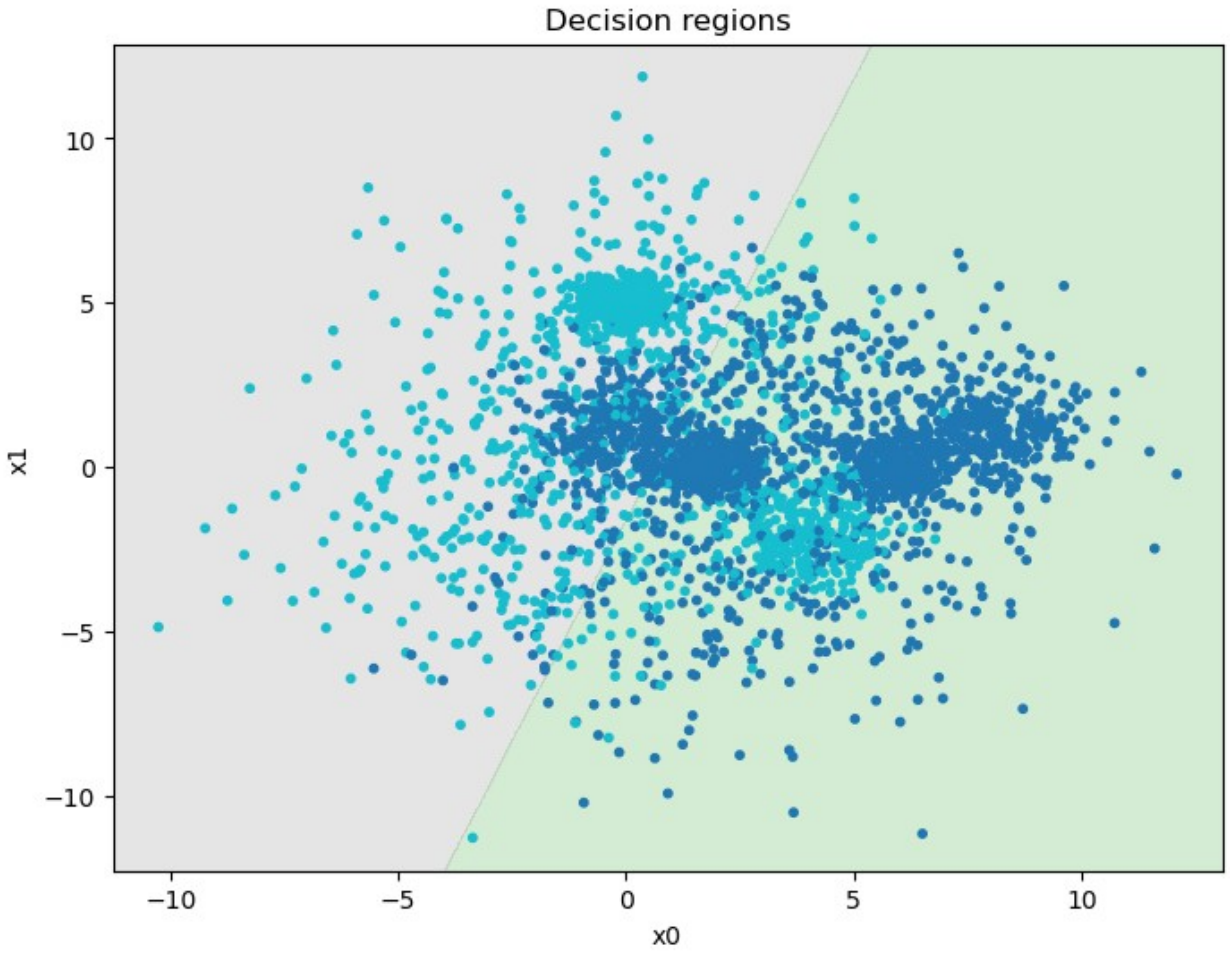
Also report precision and recall for class 1.

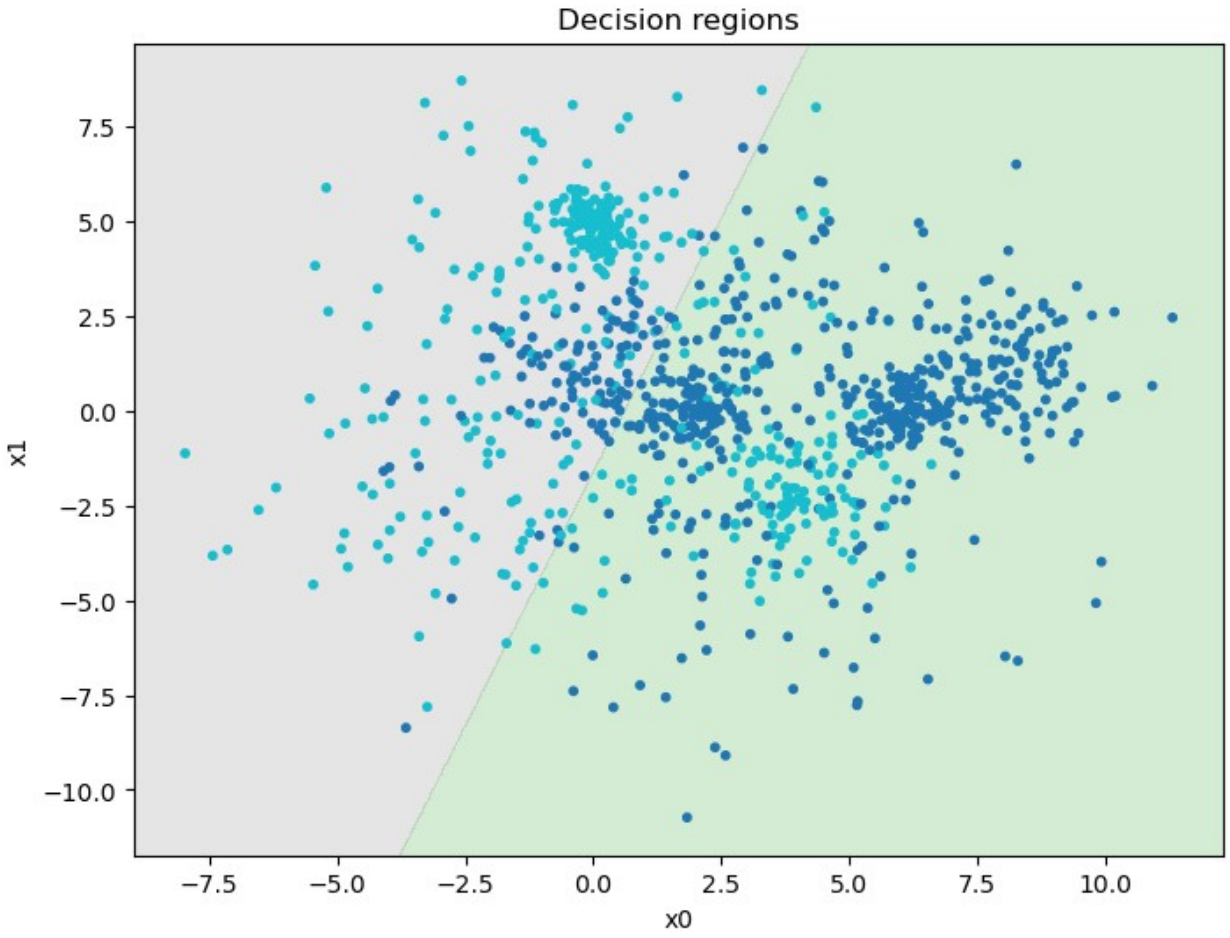
```
# My settings
epochs = 3000
learning_rate = 0.03

# Creating new objects
cl_lin = NumpyLinRegClass()
cl_log = NumpyLogRegClass()
cl_mlp = MLPBinaryLinRegClass()

# Training models
cl_lin.fit(X_train, t2_train, lr=learning_rate, epochs=epochs)
cl_log.fit(X_train, t2_train, lr=learning_rate, epochs=epochs)
cl_mlp.fit(X_train, t2_train, lr=learning_rate, epochs=epochs)

# Predicting outcomes
ac_lin_t = accuracy(cl_lin.predict(X_train), t2_train)
ac_lin_v = accuracy(cl_lin.predict(X_val), t2_val)
ac_lin_f = accuracy(cl_lin.predict(X_test), t2_test)

ac_log_t = accuracy(cl_log.predict(X_train), t2_train)
ac_log_v = accuracy(cl_log.predict(X_val), t2_val)
ac_log_f = accuracy(cl_log.predict(X_test), t2_test)

ac_mlp_t = accuracy(cl_mlp.predict(X_train), t2_train)
ac_mlp_v = accuracy(cl_mlp.predict(X_val), t2_val)
ac_mlp_f = accuracy(cl_mlp.predict(X_test), t2_test)

# Printing accuracies in a 3x3 table
print(f"Accuracy / Model | Linear Regression Perceptron | Logistic
Regression Perceptron | Multilayer Linear Regression Perceptron")
print(f"         X_train | {str(round(ac_lin_t, 4)).center(28)} |
{str(round(ac_log_t, 4)).center(30)} | {str(round(ac_mlp_t,
4)).center(39)}")
```

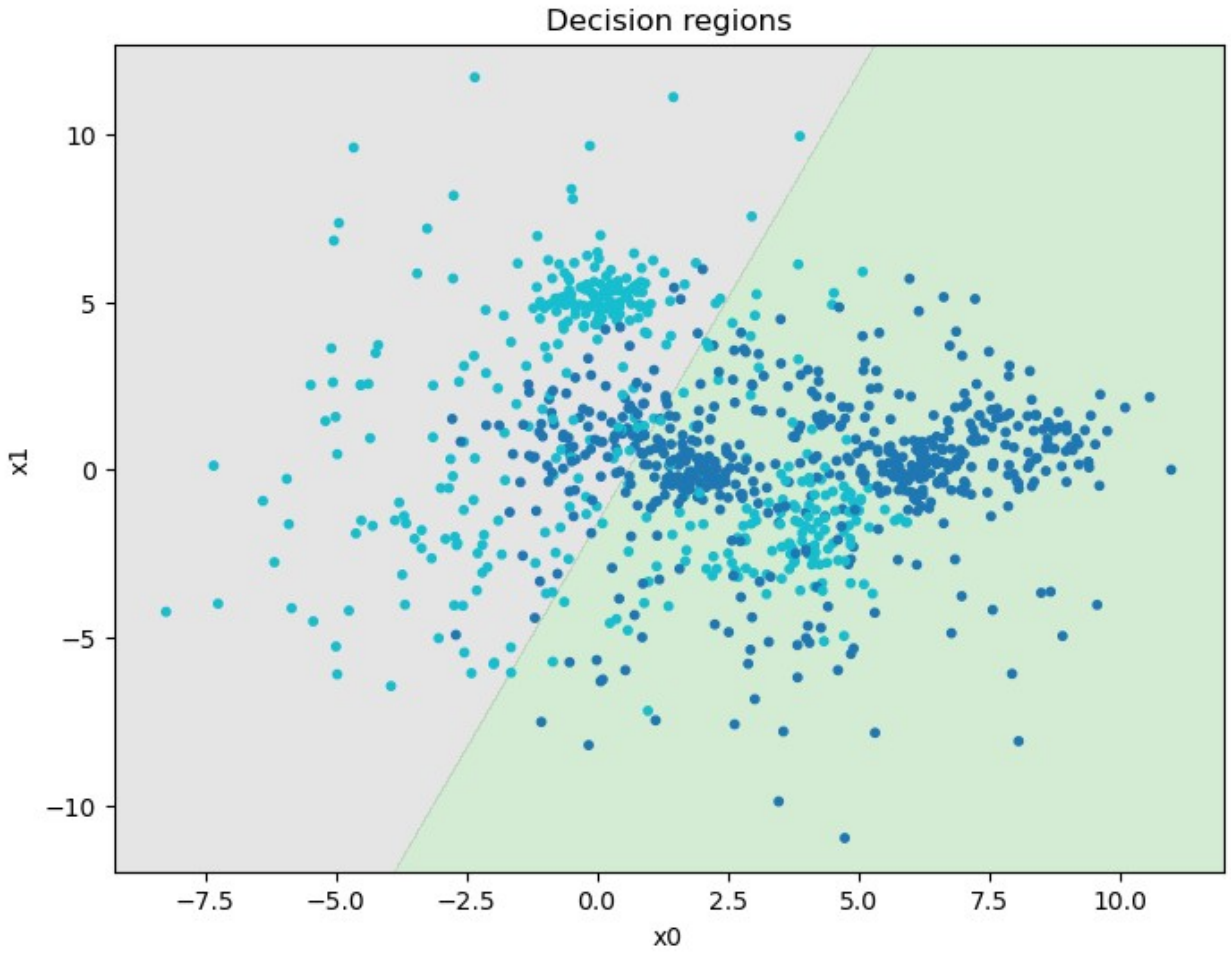
```
print(f"           X_val | {str(round(ac_lin_v, 4)).center(28)} |
{str(round(ac_log_v, 4)).center(30)} | {str(round(ac_mlp_v,
4)).center(39)}")
print(f"          X_test | {str(round(ac_lin_f, 4)).center(28)} |
{str(round(ac_log_f, 4)).center(30)} | {str(round(ac_mlp_f,
4)).center(39)}")

# Plotting results
print(f"This is the Linear Regression Perceptron - Train data,
Validation data, and Test data")
plot_decision_regions(X_train, t2_train, cl_lin)
plot_decision_regions(X_val, t2_val, cl_lin)
plot_decision_regions(X_test, t2_test, cl_lin)

print(f"This is the Logistic Regression Perceptron - Train data,
Validation data, and Test data")
plot_decision_regions(X_train, t2_train, cl_log)
plot_decision_regions(X_val, t2_val, cl_log)
plot_decision_regions(X_test, t2_test, cl_log)

print(f"This is the Multilayer Linear Regression Perceptron - Train
data, Validation data, and Test data")
plot_decision_regions(X_train, t2_train, cl_mlp)
plot_decision_regions(X_val, t2_val, cl_mlp)
plot_decision_regions(X_test, t2_test, cl_mlp)

# Discussion: It performed so well, I am shocked
# that is considering that I was getting only failures
# from all of those at some point, jesus are they bad.

# Apparently they clutched. Last chance to prove themselves
# and they did not dissapoint me, they delivered :D
# Could have been way better though...

# Still no idea what is happening with NumpyLogRegMultiClass()
```

```
C:\Users\Justa\AppData\Local\Temp\ipykernel_29000\55995779.py:23:
RuntimeWarning: overflow encountered in power
  y = 1 / (1 + np.e ** -(X_train @ self.weights))     # Sigmoid
function from lectures
C:\Users\Justa\AppData\Local\Temp\ipykernel_29000\702837997.py:3:
RuntimeWarning: overflow encountered in exp
  return 1/(1+np.exp(-x))

Accuracy / Model | Linear Regression Perceptron | Logistic Regression
Perceptron | Multilayer Linear Regression Perceptron
         X_train |            0.749             |             0.762
|                  0.796
           X_val |            0.755             |             0.771
|                  0.787
```

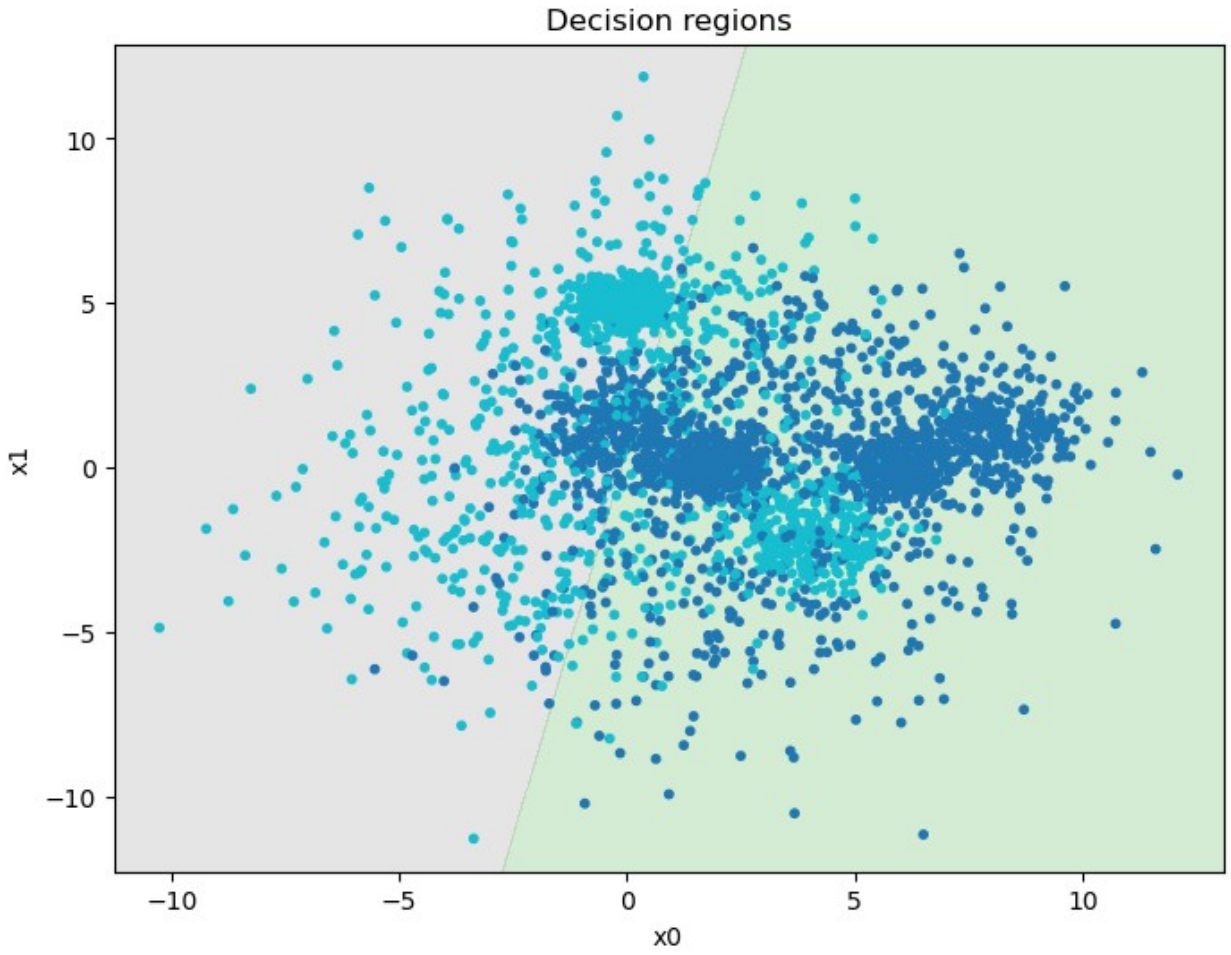
```
          X_test |            0.752              |              0.761
|                  0.784
This is the Linear Regression Perceptron - Train data, Validation
data, and Test data
```

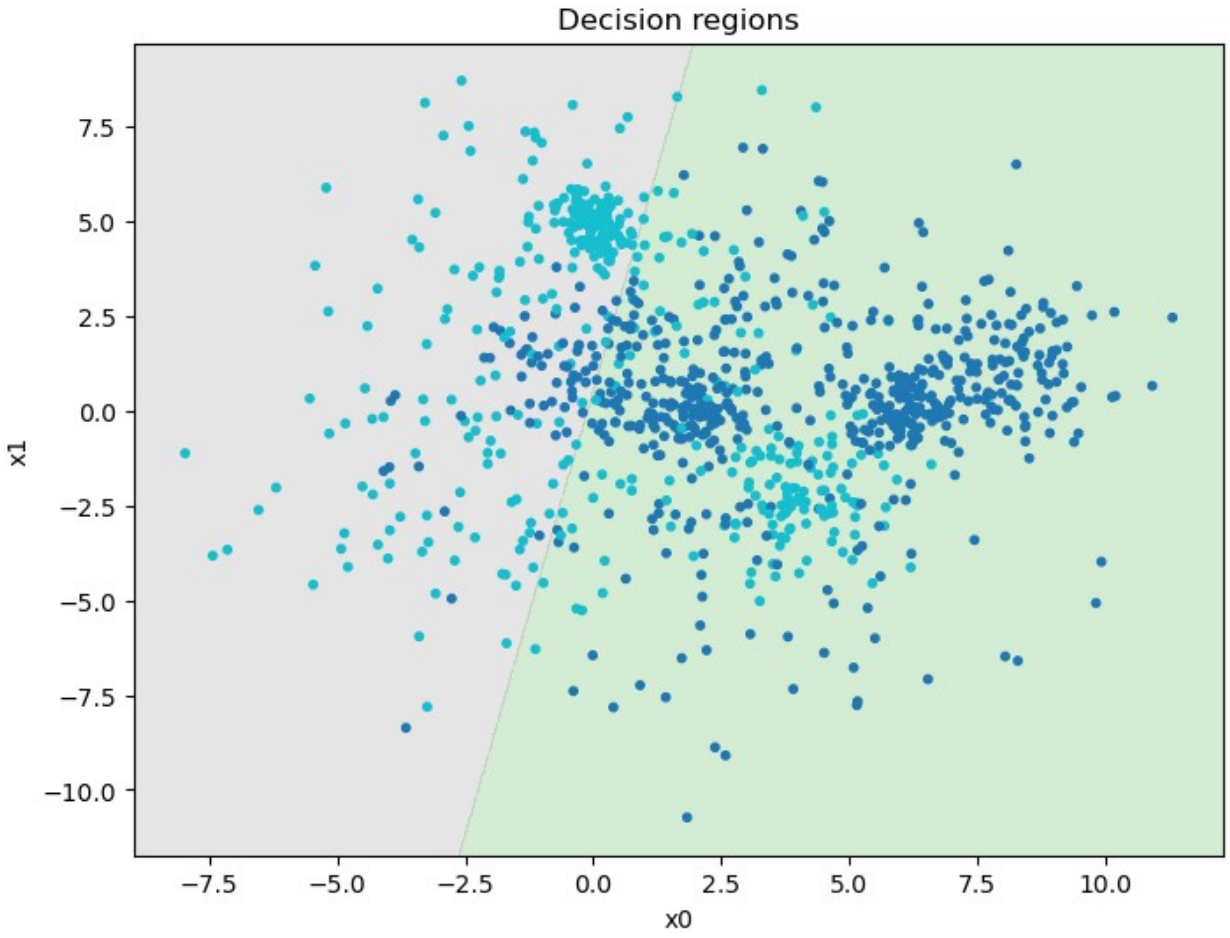


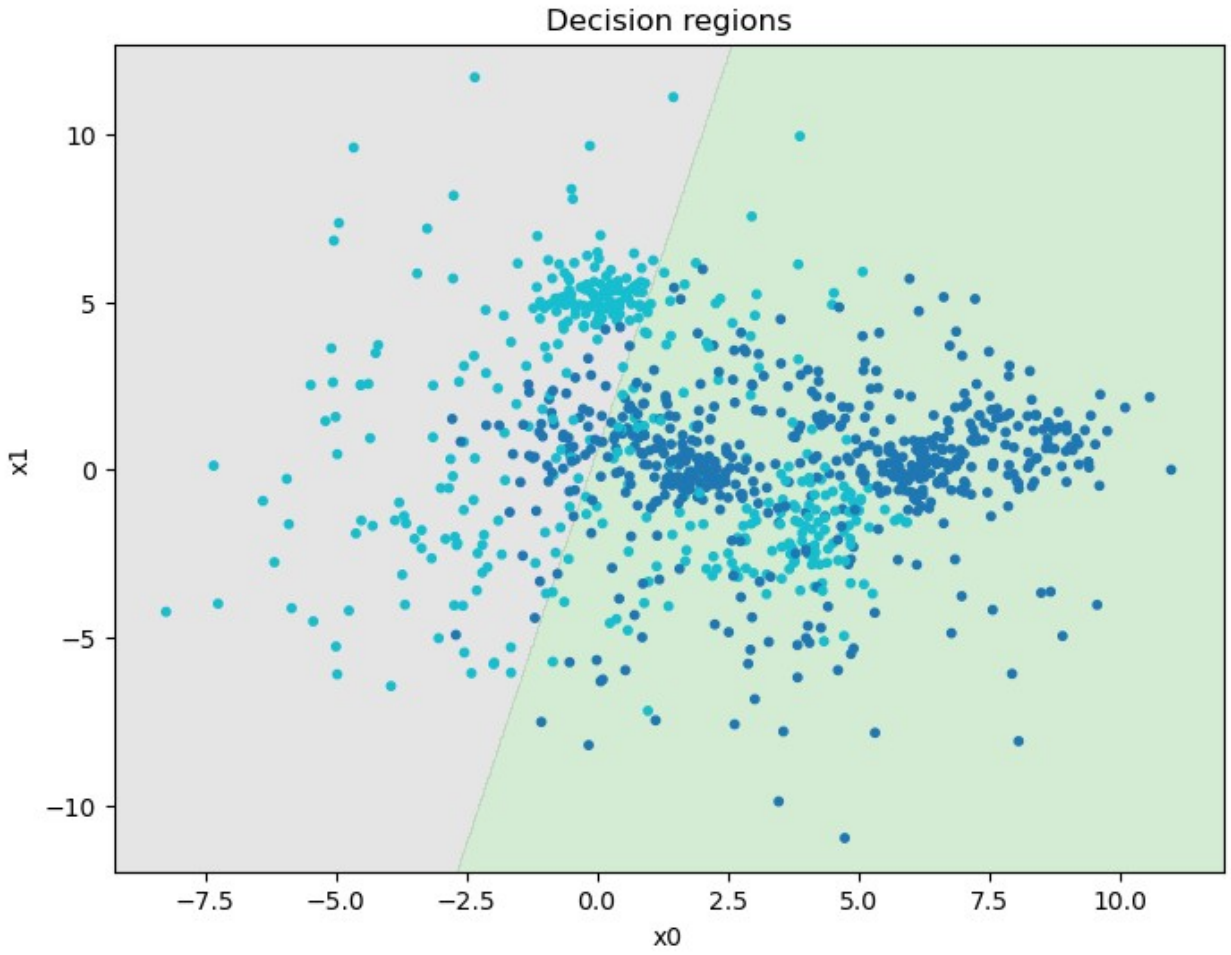

Decision regions
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
x1
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
x0



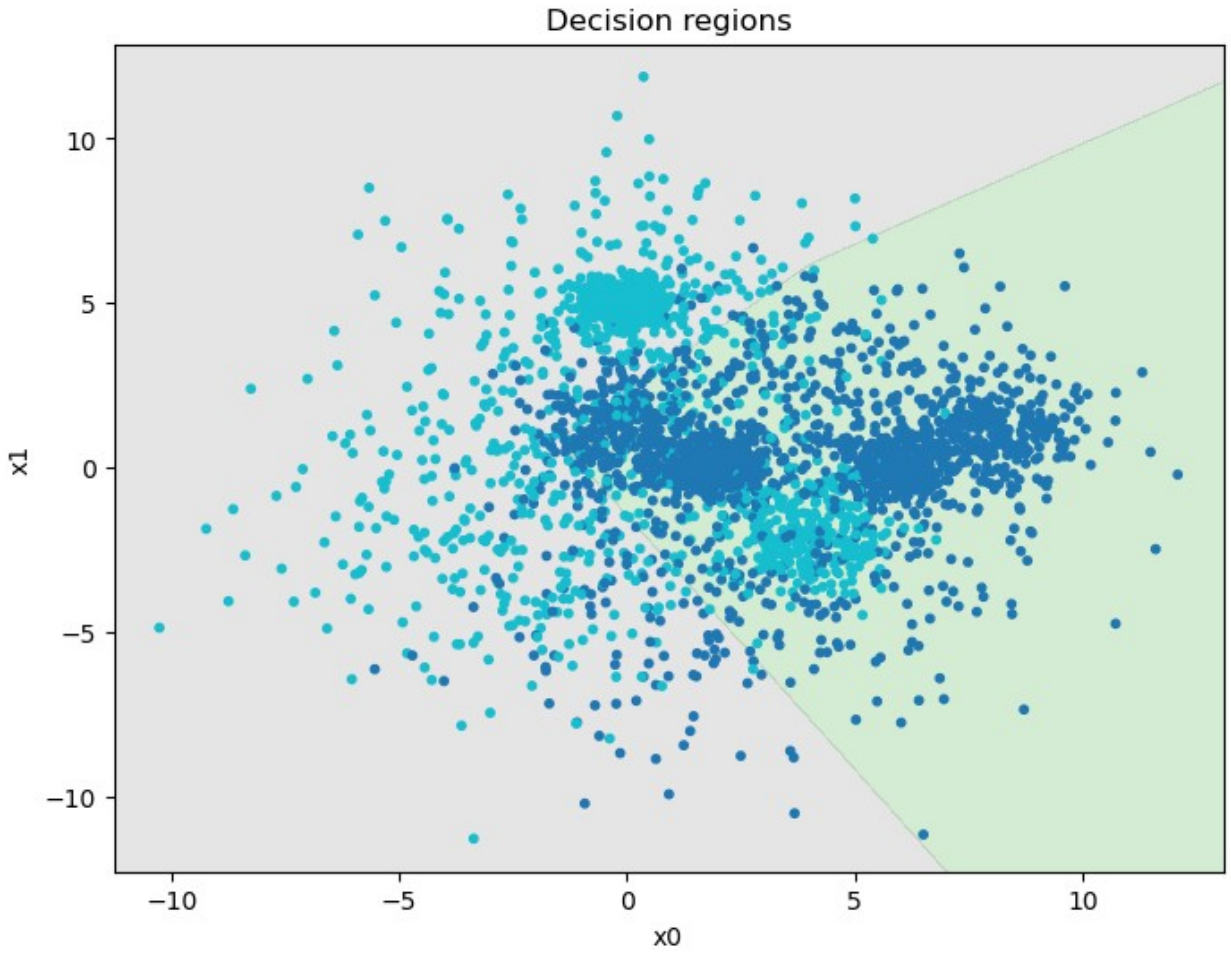

```
This is the Logistic Regression Perceptron - Train data, Validation
data, and Test data
```

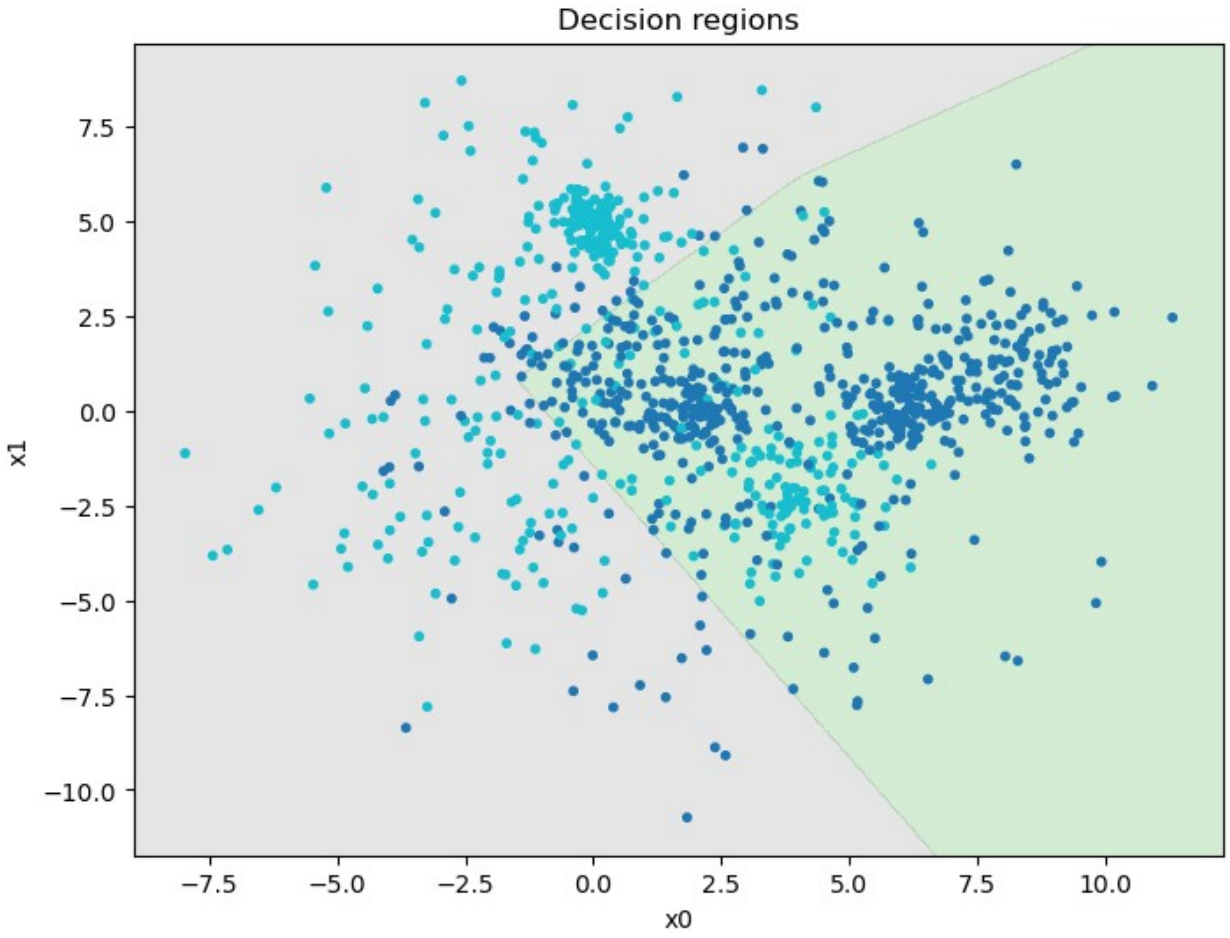

Decision regions
10
5
0
−5
−10
x1
−10
−5
0
5
10
x0

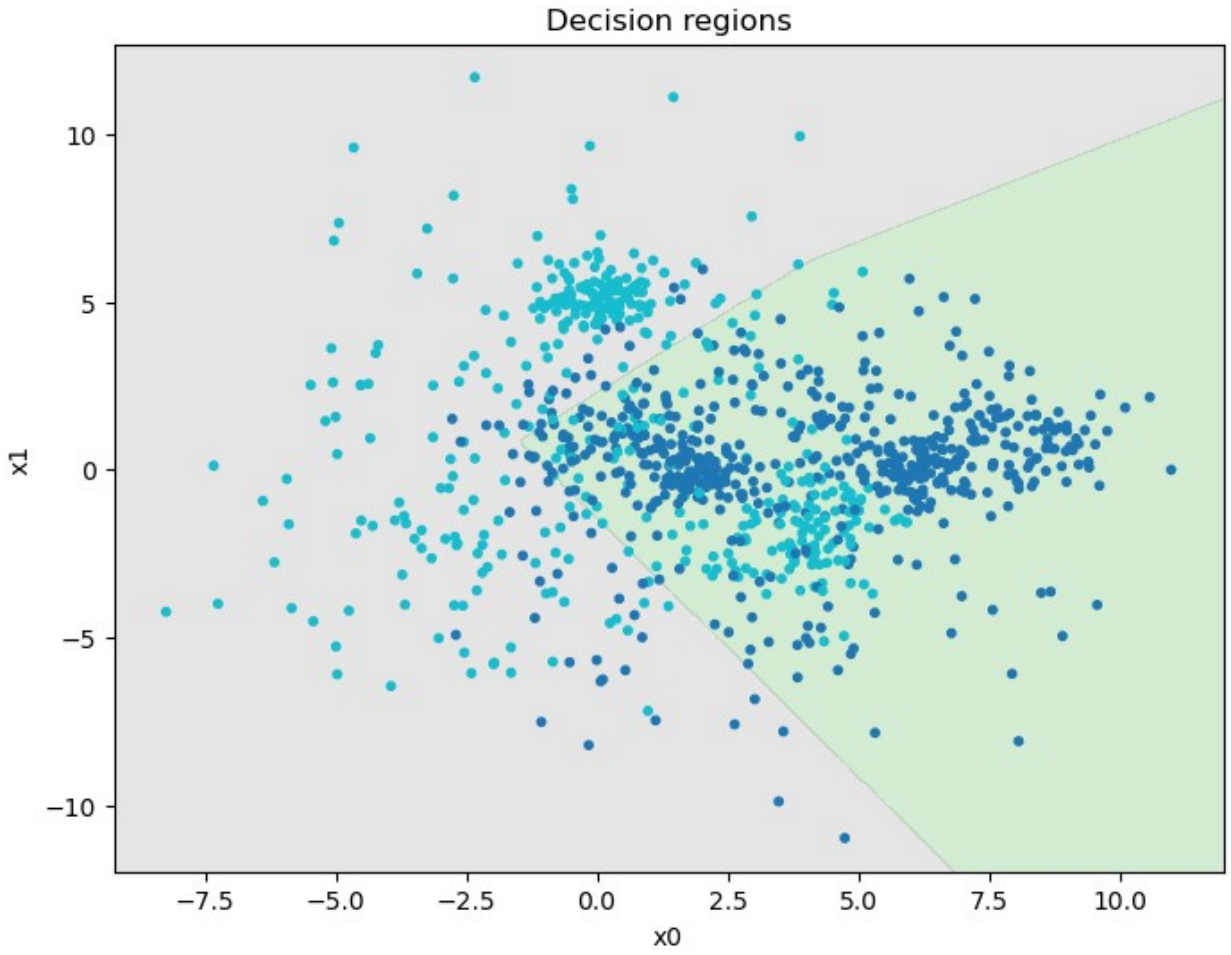

Decision regions
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
x1
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
x0




```
This is the Multilayer Linear Regression Perceptron - Train data,
Validation data, and Test data
```


Decision regions
10
5
0
−5
−10
x1
−10
−5
0
5
10
x0


Decision regions
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
x1
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
x0



For IN4050-students: Multi-class task (X, t_multi)

The following part is only mandatory for IN4050-students. IN3050-students are also welcome to give it a try though.

Compare the three multi-class classifiers: the standard multi-class and the multinomial logistic regression from part one and the multi-class neural network from part two. Evaluate on test, validation and training set as above.

Comment on the results.